

CENTRO UNIVERSITÁRIO UNIFECAP

Eduardo Souza Mattos

SISTEMA DE CONTROLE DE ESTOQUE

Documentação Teórica e Modelagem da Aplicação

São Paulo
2026

Eduardo Souza Mattos

SISTEMA DE CONTROLE DE ESTOQUE

Documentação Teórica e Modelagem da Aplicação

Trabalho acadêmico apresentado à disciplina de Development with Python, do Centro Universitário UniFECAF, como requisito parcial de avaliação da unidade curricular.

R.A.: 35984

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO.....	4
2 MODELAGEM DA APLICAÇÃO.....	6
2.1 Estrutura de dados, entidades e relacionamentos.....	6
2.2 Descrição das entidades.....	7
2.3 Relacionamentos.....	7
2.4 Decisões técnicas de modelagem.....	8
2.5 Arquitetura de duas interfaces (Desktop e Web).....	8
3 FLUXOGRAMA DA LÓGICA DA APLICAÇÃO.....	10
3.1 Descrição dos principais processos.....	10
4 REFLEXÕES SOBRE O DESAFIO PROPOSTO.....	12
4.1 Por que desenvolver um sistema de controle de estoque?.....	12
4.2 Qual o principal benefício ao digitalizar o controle de estoque?.....	12
4.3 Quais são os prós e contras enfrentados?.....	12
4.4 Quais são as barreiras ou desafios para implementar um sistema?.....	12
5 CONSIDERAÇÕES FINAIS.....	14
REFERÊNCIAS.....	15
APÊNDICE A – CÓDIGO-FONTE DA APLICAÇÃO.....	16
APÊNDICE B – SCRIPT SQL DA ESTRUTURA DO BANCO DE DADOS.....	54
APÊNDICE C – SINCRONIZAÇÃO EM TEMPO REAL (NÍVEL 2): DESAFIOS DE IMPLEMENTAÇÃO.....	56
APÊNDICE D – CAPTURAS DE TELA DA APLICAÇÃO EM EXECUÇÃO.....	59

1 INTRODUÇÃO

O controle de estoque é um dos processos mais críticos para a saúde operacional e financeira de uma empresa. Sua ausência, ou a sua execução de forma manual e desorganizada, gera falhas como ruptura de estoque, excesso de produtos parados, divergências de inventário e prejuízos financeiros. A digitalização desse processo, por meio de uma aplicação desenvolvida em Python, permite monitorar em tempo real a entrada e a saída de produtos, controlar níveis mínimos, reduzir erros humanos de lançamento e fornecer relatórios confiáveis para a tomada de decisão. Segundo Martelli e Dandaro (2015), a gestão de estoques figura entre os fatores de maior relevância para a administração de uma organização, pois envolve decisões sobre o que deve permanecer em estoque, quando reabastecê-lo e em qual quantidade, de modo que um planejamento inadequado nesse processo compromete diretamente o resultado financeiro do negócio.

Este trabalho tem como objetivo desenvolver uma aplicação de controle de estoque que permita cadastrar produtos, registrar movimentações de entrada e saída, validar os dados informados pelo usuário e disponibilizar uma interface simples e segura, com autenticação por login e senha e diferenciação de perfis de acesso (Administrador e Comum). O enunciado do desafio prevê Tkinter e Flask como bibliotecas alternativas para a integração da interface com o back-end; neste projeto, ambas foram implementadas: uma interface desktop (Tkinter) e uma interface web (Flask), compartilhando o mesmo banco de dados e as mesmas regras de negócio, de modo que uma alteração feita em uma interface é refletida na outra.

1.1 Funcionalidades da aplicação desenvolvida

- a) autenticação de usuários (login e senha) com controle de sessão;
- b) dois perfis de acesso: Administrador (cadastra usuários, produtos, categorias e fornecedores) e Comum (realiza movimentações e consultas);
- c) cadastro, edição e exclusão de produtos, com campos obrigatórios e validação de tipo de dado;
- d) registro de movimentações de entrada e saída, atualizando a quantidade em estoque e preservando o preço unitário praticado no momento;
- e) listagem do estoque atual com destaque visual para produtos abaixo da quantidade mínima;
- f) persistência dos dados em banco de dados relacional (SQLite);
- g) criptografia da senha do usuário como reforço de segurança (diferencial);

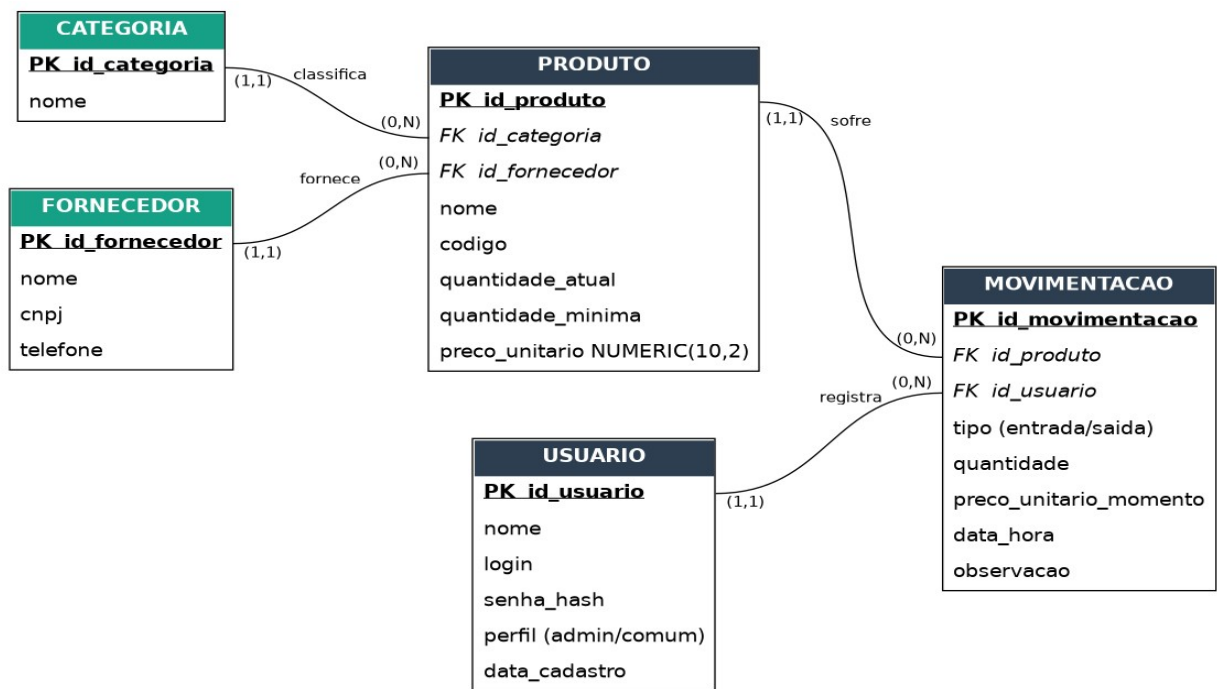
- h) geração de relatórios exportáveis (CSV ou Excel) do estoque atual e do histórico de movimentações, utilizando a biblioteca pandas;
- i) duas interfaces de acesso — desktop (Tkinter) e web (Flask) —, sincronizadas por meio do mesmo banco de dados: uma alteração feita em uma interface é refletida na outra ao atualizar/recarregar a tela.

2 MODELAGEM DA APLICAÇÃO

A aplicação foi modelada utilizando um modelo relacional composto por cinco entidades: USUARIO, CATEGORIA, FORNECEDOR, PRODUTO e MOVIMENTACAO. As entidades CATEGORIA e FORNECEDOR foram extraídas como tabelas próprias (processo de normalização), evitando que o mesmo tipo de produto ou fornecedor fosse cadastrado de formas inconsistentes dentro de um campo de texto livre. A entidade MOVIMENTACAO funciona como uma tabela associativa que registra todo evento de entrada ou saída de produto, relacionando qual usuário executou a ação, qual produto foi afetado, a quantidade movimentada, o preço unitário praticado naquele momento e a data/hora do evento — garantindo rastreabilidade completa do estoque, inclusive do histórico de preços.

2.1 Estrutura de dados, entidades e relacionamentos

Figura 1 — Modelo Entidade-Relacionamento (ER) da aplicação de controle de estoque



Fonte: elaborado pelo autor (2026).

A notação utilizada segue o padrão de cardinalidade mínima e máxima (mín,máx), indicada em cada extremidade do relacionamento: (1,1) significa que a ocorrência é obrigatória e única (exatamente um); (0,N) significa que a ocorrência é opcional e pode se repetir (zero ou muitos). As chaves primárias (PK) aparecem sublinhadas e em negrito, e as chaves estrangeiras (FK) aparecem em itálico, dentro de cada tabela.

2.2 Descrição das entidades

- a) USUARIO: chave primária id_usuario. Armazena os dados de acesso e o perfil de cada usuário (Administrador ou Comum). A senha é armazenada como hash, nunca em texto puro;
- b) CATEGORIA: chave primária id_categoria. Tabela de apoio que padroniza os tipos de produto, evitando inconsistências como "Ferragens" e "ferragens" cadastradas como categorias diferentes;
- c) FORNECEDOR: chave primária id_fornecedor. Armazena os dados das empresas fornecedoras (nome, CNPJ único e telefone), permitindo relacionar vários produtos ao mesmo fornecedor;
- d) PRODUTO: chave primária id_produto; chaves estrangeiras id_categoria (referencia CATEGORIA) e id_fornecedor (referencia FORNECEDOR). Armazena os dados cadastrais de cada item do estoque, incluindo a quantidade atual e a quantidade mínima (usada para alertas);
- e) MOVIMENTACAO: chave primária id_movimentacao; chaves estrangeiras id_produto (referencia PRODUTO) e id_usuario (referencia USUARIO). Registra cada entrada ou saída de produto, o tipo de movimento, a quantidade, o preço unitário praticado naquele momento e a data/hora.

2.3 Relacionamentos

- a) CATEGORIA (1,1) — PRODUTO (0,N): cada produto está associado a, no máximo, uma categoria (cardinalidade mínima 0, pois a categoria é opcional); uma mesma categoria pode classificar zero ou vários produtos;
- b) FORNECEDOR (1,1) — PRODUTO (0,N): cada produto tem, no máximo, um fornecedor principal (também opcional); um mesmo fornecedor pode fornecer zero ou vários produtos;
- c) USUARIO (1,1) — MOVIMENTACAO (0,N): toda movimentação pertence a exatamente um usuário (cardinalidade mínima 1, campo obrigatório); um usuário pode ter registrado zero ou várias movimentações;
- d) PRODUTO (1,1) — MOVIMENTACAO (0,N): toda movimentação pertence a exatamente um produto (obrigatório); um produto pode ter zero ou várias movimentações ao longo do tempo;

- e) em todos os casos acima, o lado "(1,1)" corresponde à entidade "um" da relação (ex.: USUARIO, PRODUTO, CATEGORIA, FORNECEDOR) e o lado "(0,N)" corresponde à entidade "muitos" (MOVIMENTACAO ou PRODUTO, conforme o caso), caracterizando relacionamentos do tipo 1:N implementados por chave estrangeira (FK) na tabela do lado "N".

2.4 Decisões técnicas de modelagem

- a) preço unitário histórico: a coluna preco_unitario_momento em MOVIMENTACAO preserva o valor praticado no exato instante da movimentação, mesmo que o preço do produto seja alterado posteriormente;
- b) precisão em valores monetários: o campo preco_unitario foi definido como NUMERIC(10,2), reduzindo o risco de erros de arredondamento em cálculos financeiros;
- c) normalização de categoria e fornecedor: extrair essas informações para tabelas próprias reduz redundância e inconsistência de dados, seguindo boas práticas de modelagem relacional;
- d) arquitetura em camadas ("Classe Modelo"): seguindo o mesmo princípio de separação entre dados e regras de negócio estudado na Unidade IV da disciplina (onde uma aplicação Flask organiza uma "Classe Modelo" para representar cada entidade do banco), este projeto adota a mesma ideia na pasta models/ (usuario.py, categoria.py, fornecedor.py, produto.py, relatorio.py): cada arquivo concentra as regras de negócio e o acesso ao banco de uma entidade específica, mantendo as camadas de interface (ui/ e webapp.py) livres de lógica de dados. É justamente essa separação que torna possível reaproveitar os mesmos models/ tanto na interface desktop (Tkinter) quanto na interface web (Flask), sem duplicar nenhuma regra de negócio.

2.5 Arquitetura de duas interfaces (Desktop e Web)

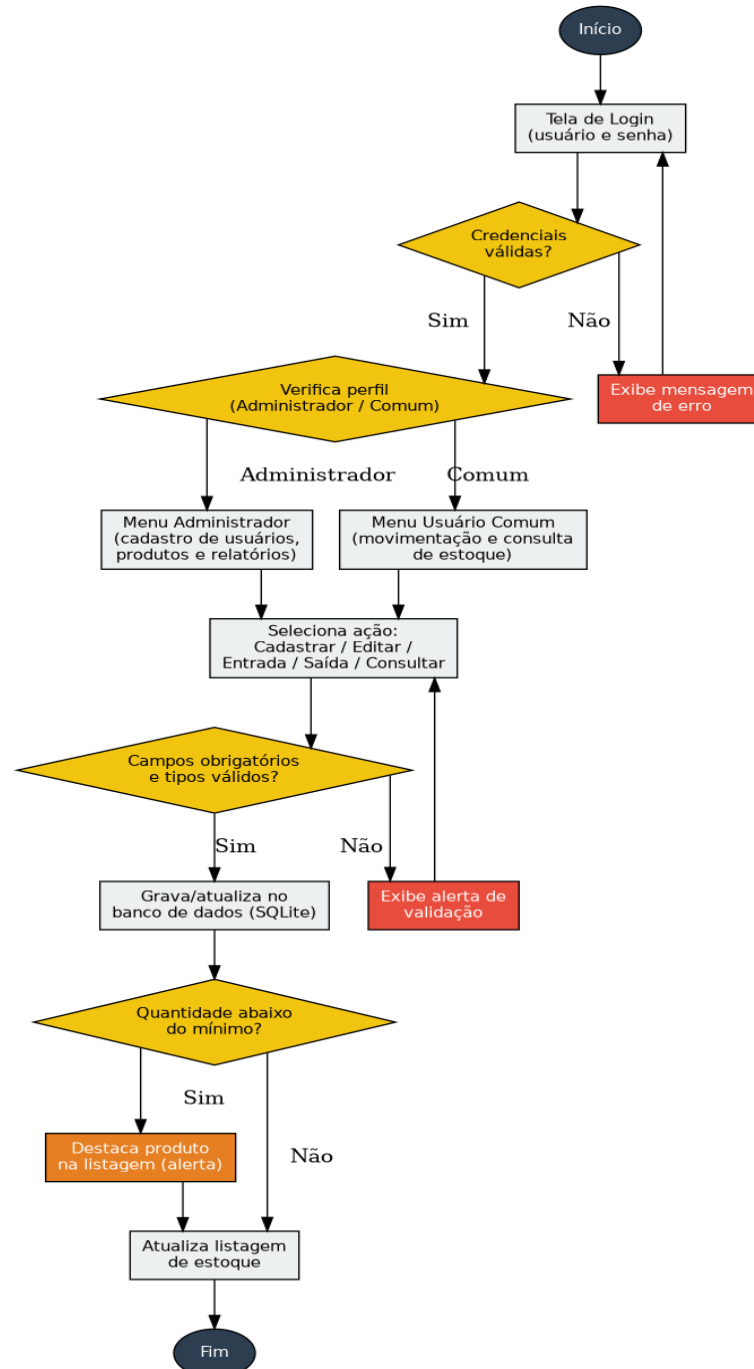
O enunciado do desafio prevê Tkinter e Flask como alternativas de biblioteca para integrar a interface com o back-end (item "g"). Neste projeto, as duas foram implementadas simultaneamente: main.py inicia a interface desktop (Tkinter), enquanto webapp.py inicia a interface web (Flask), acessível pelo navegador em <http://127.0.0.1:5000>. As duas interfaces são processos independentes, mas chamam exatamente as mesmas funções da camada models/, que sempre leem e gravam diretamente no arquivo database/estoque.db — não há nenhuma cópia de dados mantida em memória entre elas.

Essa arquitetura compartilhada é o que permite a sincronização entre as duas interfaces: um produto cadastrado pelo Tkinter já pode ser consultado pela interface web, e vice-versa. A sincronização implementada é "sob demanda": no Tkinter, isso ocorre ao clicar no botão "Atualizar Lista"; no Flask, ocorre automaticamente a cada requisição HTTP, ou seja, toda vez que a página é carregada ou recarregada (F5) no navegador. Uma sincronização em tempo real, sem a necessidade de recarregar a tela, exigiria mecanismos adicionais — como WebSockets (por exemplo, com a extensão Flask-SocketIO) ou consultas repetidas via JavaScript (polling) —, que ficaram definidos como melhoria futura, por ampliarem significativamente a complexidade da solução sem alterar o atendimento aos requisitos do desafio.

3 FLUXOGRAMA DA LÓGICA DA APLICAÇÃO

O fluxograma a seguir descreve o funcionamento geral do sistema, desde a autenticação do usuário até a atualização do estoque e a geração de alertas de quantidade mínima.

Figura 2 — Fluxograma da lógica da aplicação de controle de estoque



Fonte: elaborado pelo autor (2026).

3.1 Descrição dos principais processos

- a) autenticação: o usuário informa login e senha; o sistema valida as credenciais e identifica o perfil de acesso;

- b) cadastro: administradores podem cadastrar novos usuários, e qualquer usuário pode cadastrar produtos, categorias e fornecedores, com validação de campos obrigatórios e de tipo;
- c) movimentação: qualquer usuário autenticado pode registrar entrada ou saída de produtos, o que atualiza a quantidade em estoque e registra o preço praticado no momento;
- d) alertas: após cada atualização, o sistema verifica se a quantidade do produto ficou abaixo do mínimo configurado e destaca o item na listagem;
- e) relatórios: além da listagem de estoque atualizada em tempo real (que funciona como um relatório instantâneo em tela), o usuário pode gerar e exportar relatórios em CSV ou Excel — tanto do estoque atual quanto do histórico completo de movimentações —, usando a biblioteca pandas para montar e salvar os dados no formato escolhido;
- f) acesso multi-interface: qualquer um dos processos acima (cadastro, movimentação, alertas ou relatórios) pode ser realizado tanto pela interface desktop quanto pela interface web, já que ambas chamam as mesmas funções de negócio sobre o mesmo banco de dados.

4 REFLEXÕES SOBRE O DESAFIO PROPOSTO

4.1 Por que desenvolver um sistema de controle de estoque?

Uma empresa contrata ou desenvolve um sistema de controle de estoque para eliminar a dependência de processos manuais, que são lentos e suscetíveis a erros humanos, como contagens incorretas, perda de planilhas ou lançamentos duplicados. Fatores como a necessidade de evitar ruptura de estoque, o excesso de capital parado em mercadorias sem giro, e a exigência de dados confiáveis para negociar com fornecedores e planejar compras futuras são determinantes para essa decisão. Em resumo, o sistema resolve um ponto crítico do processo: transformar dados dispersos e pouco confiáveis em informação organizada e disponível em tempo real.

4.2 Qual o principal benefício ao digitalizar o controle de estoque?

O principal benefício não está apenas na velocidade da busca, mas também no ganho de confiabilidade das informações e na redução de impactos operacionais e ambientais. Ao substituir formulários e papéis assinados manualmente por registros digitais de movimentação, a empresa reduz o consumo de papel, agiliza auditorias e cria um histórico rastreável de quem realizou cada entrada ou saída. Esse conjunto de ganhos permite decisões mais rápidas e embasadas, como reposição automática de itens críticos e identificação de produtos parados.

4.3 Quais são os prós e contras enfrentados?

Como pontos positivos, destacam-se: maior controle sobre entradas e saídas, redução de erros de digitação por meio de validações, geração de alertas automáticos para produtos abaixo do mínimo, segurança de acesso por meio de login e perfis diferenciados, e histórico completo de movimentações para auditoria. Como pontos negativos, o sistema exige investimento inicial de tempo de implantação e treinamento de usuários, pode gerar resistência dos colaboradores acostumados ao processo manual, e depende da correta digitação dos dados — se a entrada de informação for feita de forma errada, o sistema reproduzirá esse erro nos relatórios. Filho (2025) observa que, de modo geral, ferramentas informatizadas de controle de estoque — sejam planilhas eletrônicas, leitores de código de barras ou sistemas mais completos de gestão de armazém — reduzem substancialmente erros humanos e o tempo gasto em contagens manuais, mas seu retorno depende diretamente da correta parametrização inicial e da adesão dos colaboradores ao novo processo. Ainda assim, os ganhos de eficiência, segurança e rastreabilidade tendem a superar essas desvantagens ao longo do tempo.

4.4 Quais são as barreiras ou desafios para implementar um sistema?

As barreiras vão desde o nível estratégico até o operacional. No nível estratégico, a diretoria pode relutar em autorizar o investimento inicial, priorizando economia imediata em vez de ganho de eficiência a médio prazo. No nível operacional, a principal barreira costuma ser a resistência dos funcionários à mudança cultural de trabalho, que podem enxergar o sistema como algo que aumenta a fiscalização sobre o próprio trabalho. Some-se a isso desafios técnicos, como garantir a segurança dos dados armazenados e assegurar que a aplicação seja intuitiva o suficiente para reduzir a curva de aprendizado. A solução apresentada neste trabalho busca mitigar esses pontos oferecendo uma interface simples, validações de campo, controle de acesso por perfil e alertas visuais automáticos.

5 CONSIDERAÇÕES FINAIS

O desenvolvimento desta aplicação evidenciou a importância de planejar a modelagem de dados antes da implementação do código, pois a definição clara das entidades e relacionamentos evita retrabalho durante o desenvolvimento. Entre os principais desafios técnicos enfrentados estão: garantir a validação consistente dos dados informados pelo usuário, controlar corretamente a sessão e o perfil de acesso, e manter a integridade referencial entre produtos, categorias, fornecedores, usuários e movimentações.

Como decisões técnicas, optou-se pelo uso do SQLite como banco de dados por sua simplicidade de implantação em um projeto acadêmico, pelas bibliotecas Tkinter e Flask para as duas interfaces (desktop e web), e pela biblioteca pandas para a geração dos relatórios exportáveis (CSV/Excel), conteúdo estudado na Unidade III da disciplina. A modelagem de dados também foi revisada à luz de boas práticas de projetos de modelagem relacional voltados a controle de vendas e estoque, como a normalização de categoria e fornecedor em tabelas próprias e o registro do preço histórico em cada movimentação, o que tornou o modelo mais robusto sem ampliar o escopo funcional do sistema. A arquitetura em camadas do projeto (banco de dados, regras de negócio e interface) também dialoga diretamente com o conceito de "Classe Modelo" apresentado na Unidade IV, e foi o que tornou viável reaproveitar a mesma camada de regras de negócio (models/) tanto na interface Tkinter quanto na interface Flask, sem duplicação de código — ambas as bibliotecas de interface previstas como alternativas no desafio proposto foram, neste projeto, implementadas simultaneamente e de forma sincronizada. Como melhoria futura, destaca-se a possibilidade de evoluir essa sincronização para tempo real (com WebSockets), a implementação de relatórios gráficos de movimentação e a importação de produtos em lote a partir de arquivos CSV, também abordada na Unidade III.

Durante os testes de execução da interface web, foi enfrentado um problema operacional real: o endereço `http://127.0.0.1:5000` deixou de responder no navegador após uma execução anterior do servidor Flask não ter sido encerrada corretamente, mantendo a porta 5000 ocupada em segundo plano. O diagnóstico foi feito com o comando `lsof -i :5000`, que identificou o processo remanescente, encerrado em seguida com `kill -9 <PID>`, permitindo que o servidor fosse iniciado normalmente na sequência. Esse episódio reforçou, na prática, a importância de encerrar corretamente processos de servidores de desenvolvimento (com Ctrl+C) antes de fechar o terminal, e está documentado, junto de outras soluções de problemas comuns, no arquivo README.md entregue com o projeto.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6024: informação e documentação: numeração progressiva das seções de um documento. Rio de Janeiro: ABNT, 2012.

CENTRO UNIVERSITÁRIO UNIFECAP. Development with Python: Unidade I. São Paulo: UniFECAF, 2026. Material didático da disciplina Development with Python.

CENTRO UNIVERSITÁRIO UNIFECAP. Development with Python: Unidade II. São Paulo: UniFECAF, 2026. Material didático da disciplina Development with Python.

CENTRO UNIVERSITÁRIO UNIFECAP. Development with Python: Unidade III. São Paulo: UniFECAF, 2026. Material didático da disciplina Development with Python.

CENTRO UNIVERSITÁRIO UNIFECAP. Development with Python: Unidade IV. São Paulo: UniFECAF, 2026. Material didático da disciplina Development with Python.

FILHO, João Batista Chaves de Moura. Informatização do controle de estoques: vantagens e desvantagens das principais ferramentas utilizadas. *International Integrate Scientific*, v. 5, n. 51, set. 2025. Disponível em: <https://iiscientific.com/artigos/b74b2f/>. Acesso em: 07 set. 2026.

MARTELLI, Leandro Lopez; DANDARO, Fernando. Planejamento e controle de estoque nas organizações. *Revista Gestão Industrial*, Ponta Grossa, v. 11, n. 2, p. 170-185, 2015. Disponível em: <https://periodicos.utfpr.edu.br/revistagi/article/view/2733>. Acesso em: 07 set. 2026.

PANDAS DEVELOPMENT TEAM. pandas documentation. [S. l.]: NumFOCUS, [2026?]. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 12 ago. 2026.

PYTHON SOFTWARE FOUNDATION. Python 3 documentation. [S. l.]: PSF, [2026?]. Disponível em: <https://docs.python.org/3/>. Acesso em: 12 ago. 2026.

SQLITE CONSORTIUM. SQLite documentation. [S. l.]: SQLite Consortium, [2026?]. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: 12 ago. 2026.

APÊNDICE A – CÓDIGO-FONTE DA APLICAÇÃO

Este apêndice apresenta, na íntegra, o código-fonte Python da aplicação de controle de estoque descrita neste trabalho, organizado nos mesmos módulos entregues no arquivo compactado do projeto (Projeto_Python_Control_Estoque.zip), incluindo o ponto de entrada da interface web (webapp.py). Os demais templates HTML da interface Flask (formulários de produto, fornecedor, movimentação, usuário, login e relatórios) seguem a mesma estrutura de base.html e dashboard.html reproduzidos a seguir, e estão disponíveis na íntegra na pasta templates/ do arquivo compactado do projeto.

A.1 main.py

```

"""
main.py
=====
Ponto de entrada da aplicação de Controle de Estoque.

Este arquivo é propositalmente pequeno: sua única função é
"orquestrar" a ordem de inicialização do sistema, sem conter
nenhuma regra de negócio nem nenhum componente visual próprio.
Isso segue o princípio de que cada arquivo do projeto deve ter
uma responsabilidade clara (banco de dados, regra de negócio,
interface, ou – como aqui – apenas "dar a partida" no programa).

Como executar:
  1. Tenha o Python 3 instalado (https://www.python.org/)
  2. No terminal, dentro desta pasta, execute: python main.py
  3. Faça login com o usuário padrão: login "admin", senha "admin123"
=====
"""

from database.db import criar_tabelas
from ui.login import TelaLogin
from ui.principal import TelaPrincipal

def abrir_tela_principal(usuario):
    """
    Esta função é passada como "callback" para a TelaLogin (veja
    o parâmetro ao_autenticar em ui/login.py). Ela só é executada
    DEPOIS que o login foi validado com sucesso, recebendo os
    dados do usuário autenticado.

    Aqui criamos a janela principal (TelaPrincipal) e chamamos
    app.mainloop() – o "loop de eventos" do Tkinter, responsável
    por manter a janela aberta, respondendo a cliques, digitação
    etc., até que o usuário a feche.
    """
    app = TelaPrincipal(usuario)
    app.mainloop()

# -----
# Este bloco só roda quando executamos "python main.py"
# diretamente (e não quando este arquivo é importado por outro
# script, o que não deveria acontecer neste projeto, mas é uma
# boa prática do Python de qualquer forma).
# -----
if __name__ == "__main__":
    # 1) Garante que o banco de dados e as tabelas existem antes
    # de qualquer tela ser exibida (se já existirem, esta
    # chamada não faz nada, graças ao "CREATE TABLE IF NOT
    # EXISTS" usado em database/db.py).
    criar_tabelas()

    # 2) Abre a tela de login. Note que passamos a FUNÇÃO
    # abrir_tela_principal (sem executá-la agora, sem
    # parênteses) – ela só será chamada de dentro de
    # TelaLogin, no momento exato em que o login for validado.
    login_app = TelaLogin(ao_autenticar=abrir_tela_principal)

    # 3) Inicia o loop de eventos da tela de login. O programa
    # "para" nesta linha até a janela de login ser fechada
    # (seja porque o login deu certo e chamamos self.destroy())
    # dentro dela, seja porque o usuário fechou a janela no X).
    login_app.mainloop()

```

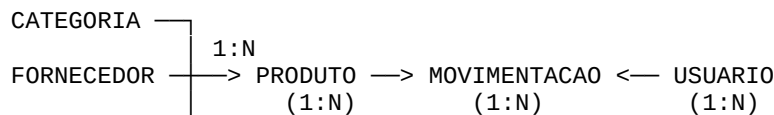

A.2 database/db.py

```
"""
database/db.py
=====
Este módulo é a "camada de banco de dados" da aplicação. Ele tem
três responsabilidades:
```

1. Definir ONDE o arquivo do banco SQLite fica salvo (DB_PATH);
2. Abrir conexões com esse banco (get_connection);
3. Criar as tabelas na primeira execução, se elas ainda não existirem (criar_tabelas).

Usamos o SQLite porque ele não exige instalação de um servidor separado: o "banco de dados" é literalmente um único arquivo (.db) salvo junto com o projeto, e o Python já sabe conversar com ele através do módulo padrão `sqlite3`.

 MODELAGEM (versão atualizada, com 5 tabelas):



- CATEGORIA e FORNECEDOR foram extraídas como tabelas próprias (normalização), em vez de texto livre dentro de PRODUTO, para evitar inconsistência de dados (ex.: "Ferragens" vs "ferragens") e permitir reaproveitar o mesmo fornecedor/categoria em vários produtos sem repetir informação.
- MOVIMENTACAO agora guarda também o preço unitário praticado no momento exato da movimentação (preco_unitario_momento), preservando o histórico de valores mesmo que o preço do produto mude depois – a mesma ideia usada em sistemas de vendas para não perder o preço "congelado" de uma transação já concluída.

```
-----
"""
import sqlite3    # módulo padrão do Python para trabalhar com SQLite (não precisa
instalar nada)
import hashlib     # módulo padrão usado para gerar o hash (criptografia) da senha
import os         # módulo padrão usado aqui só para montar o caminho do arquivo do
banco
```

```
DB_PATH = os.path.join(os.path.dirname(__file__), "estoque.db")
```

```
def get_connection():
    """
    Abre e devolve uma conexão com o banco de dados SQLite.

    - PRAGMA foreign_keys = ON: liga a validação de chaves
      estrangeiras (desligada por padrão no SQLite), garantindo
      que um PRODUTO não referencie uma CATEGORIA ou FORNECEDOR
      inexistente, e que uma MOVIMENTACAO não referencie um
      PRODUTO ou USUARIO inexistente.
    - conn.row_factory = sqlite3.Row: permite acessar cada linha
      tanto por índice quanto por nome de coluna (linha["nome"]).
    """
    conn = sqlite3.connect(DB_PATH)
    conn.execute("PRAGMA foreign_keys = ON")
    conn.row_factory = sqlite3.Row
    return conn
```

```
def hash_senha(senha: str) -> str:
```

```

"""
Gera o hash SHA-256 de uma senha em texto puro, para nunca
guardarmos a senha original no banco (requisito opcional "l.
Criptografia de senha" do enunciado).
"""
return hashlib.sha256(senha.encode("utf-8")).hexdigest()

def criar_tabelas():
    """
    Cria as cinco tabelas do sistema, caso ainda não existam, e
    garante que exista pelo menos um usuário administrador para
    o primeiro acesso.
    """
    conn = get_connection()
    cur = conn.cursor()

    # -----
    # Tabela USUARIO – quem pode acessar o sistema.
    # -----
    cur.execute("""
        CREATE TABLE IF NOT EXISTS usuario (
            id_usuario    INTEGER PRIMARY KEY AUTOINCREMENT,
            nome           TEXT NOT NULL,
            login          TEXT NOT NULL UNIQUE,
            senha_hash     TEXT NOT NULL,
            perfil         TEXT NOT NULL CHECK (perfil IN ('admin', 'comum')),
            data_cadastro  TEXT DEFAULT CURRENT_TIMESTAMP
        )
    """)

    # -----
    # Tabela CATEGORIA (NOVA)
    # Antes, "categoria" era um campo de texto livre dentro de
    # PRODUTO. Agora é uma entidade própria, evitando que o mesmo
    # tipo de produto seja cadastrado com grafias diferentes
    # ("Ferragens", "ferragens", "FERRAGENS...") em produtos
    # distintos – o que atrapalharia relatórios e filtros.
    # -----
    cur.execute("""
        CREATE TABLE IF NOT EXISTS categoria (
            id_categoria  INTEGER PRIMARY KEY AUTOINCREMENT,
            nome           TEXT NOT NULL UNIQUE
        )
    """)

    # -----
    # Tabela FORNECEDOR (NOVA)
    # Passa a existir como entidade própria, com CNPJ único
    # (identificador legal da empresa fornecedora), permitindo
    # relacionar vários produtos a um mesmo fornecedor.
    # -----
    cur.execute("""
        CREATE TABLE IF NOT EXISTS fornecedor (
            id_fornecedor INTEGER PRIMARY KEY AUTOINCREMENT,
            nome           TEXT NOT NULL,
            cnpj           TEXT NOT NULL UNIQUE,
            telefone       TEXT
        )
    """)

    # -----
    # Tabela PRODUTO (ATUALIZADA)
    # - id_categoria e id_fornecedor: chaves estrangeiras que
    #   substituem o antigo campo de texto livre "categoria".
    #   Ambas aceitam NULL, pois nem todo produto precisa ter
    #   fornecedor ou categoria definidos no momento do cadastro.
    # - preco_unitario: tipo de dado NUMERIC(10,2). O SQLite não

```

```

# impõe rigidamente esse tipo (ele usa "type affinity"),
# mas declarar NUMERIC(10,2) documenta a intenção de guardar
# um valor monetário com 2 casas decimais, e o próprio driver
# Python passa a tratar esse campo com afinidade numérica,
# evitando comportamentos inesperados de ponto flutuante
# puro (tipo REAL) em cálculos financeiros.
# -----
cur.execute("""
    CREATE TABLE IF NOT EXISTS produto (
        id_produto          INTEGER PRIMARY KEY AUTOINCREMENT,
        nome                 TEXT NOT NULL,
        codigo               TEXT NOT NULL UNIQUE,
        id_categoria         INTEGER,
        id_fornecedor        INTEGER,
        quantidade_atual     INTEGER NOT NULL DEFAULT 0,
        quantidade_minima    INTEGER NOT NULL DEFAULT 0,
        preco_unitario       NUMERIC(10,2) NOT NULL DEFAULT 0,
        FOREIGN KEY (id_categoria) REFERENCES categoria (id_categoria),
        FOREIGN KEY (id_fornecedor) REFERENCES fornecedor (id_fornecedor)
    )
""")

# -----
# Tabela MOVIMENTACAO (ATUALIZADA)
# - preco_unitario_momento (NOVO): guarda o preço unitário do
# produto exatamente no instante da movimentação. Sem essa
# coluna, se o preço do produto fosse editado no futuro, o
# histórico de movimentações antigas passaria a "mentir"
# sobre o valor praticado naquela época. Isso é o mesmo
# princípio do preço histórico em notas fiscais de venda.
# -----
cur.execute("""
    CREATE TABLE IF NOT EXISTS movimentacao (
        id_movimentacao     INTEGER PRIMARY KEY AUTOINCREMENT,
        id_produto           INTEGER NOT NULL,
        id_usuario           INTEGER NOT NULL,
        tipo                 TEXT NOT NULL CHECK (tipo IN ('entrada',
'saida'))),
        quantidade           INTEGER NOT NULL,
        preco_unitario_momento NUMERIC(10,2) NOT NULL DEFAULT 0,
        data_hora            TEXT DEFAULT CURRENT_TIMESTAMP,
        observacao           TEXT,
        FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
        FOREIGN KEY (id_usuario) REFERENCES usuario (id_usuario)
    )
""")

# -----
# Usuário administrador padrão (primeira execução).
# -----
cur.execute("SELECT COUNT(*) AS total FROM usuario")
if cur.fetchone()["total"] == 0:
    cur.execute(
        "INSERT INTO usuario (nome, login, senha_hash, perfil) VALUES (?, ?, ?,
?)",
        ("Administrador", "admin", hash_senha("admin123"), "admin"),
    )

    conn.commit()
    conn.close()

if __name__ == "__main__":
    criar_tabelas()
    print(f"Banco de dados criado/verificado em: {DB_PATH}")
    print("Usuário padrão -> login: admin | senha: admin123 (altere após o primeiro
acesso)")

```


A.3 models/categoria.py

```

"""
models/categoria.py
=====
Regras de negócio da entidade CATEGORIA, criada para normalizar
o antigo campo de texto livre que existia dentro de PRODUTO.
=====
"""

from database.db import get_connection

def listar_categorias():
    """Retorna todas as categorias cadastradas, em ordem alfabética."""
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM categoria ORDER BY nome")
    dados = cur.fetchall()
    conn.close()
    return dados

def obter_ou_criar_categoria(cur, nome: str):
    """
    Recebe o NOME de uma categoria digitado/selecionado na tela e
    devolve o id_categoria correspondente:
    - Se já existir uma categoria com esse nome (comparação sem
      diferenciar maiúsculas/minúsculas, graças a "COLLATE
      NOCASE"), reaproveita o id existente;
    - Se não existir, cria uma nova categoria na hora e devolve o
      id recém-criado (cur.lastrowid).

    Por que receber "cur" em vez de abrir sua própria conexão?
    Para que a criação da categoria aconteça DENTRO da mesma
    transação do cadastro do produto (veja cadastrar_produto em
    models/produto.py) – se o cadastro do produto falhar por
    qualquer motivo, a categoria recém-criada também não fica
    "presa" no banco sem uso.

    Se o campo vier vazio, devolvemos None (categoria é opcional).
    """
    nome = (nome or "").strip()
    if not nome:
        return None

    cur.execute("SELECT id_categoria FROM categoria WHERE nome = ? COLLATE NOCASE",
    (nome,))
    linha = cur.fetchone()
    if linha:
        return linha["id_categoria"]

    cur.execute("INSERT INTO categoria (nome) VALUES (?)", (nome,))
    return cur.lastrowid

```

A.4 models/fornecedor.py

```

"""
models/fornecedor.py
=====
Regras de negócio da entidade FORNECEDOR (nova). Diferente da
categoria, o fornecedor exige um cadastro mais completo (nome,
CNPJ e telefone), então ele é cadastrado em uma janela própria,
em vez de ser criado "na hora" a partir de um campo de texto.
=====
"""

from database.db import get_connection

def listar_fornecedores():
    """Retorna todos os fornecedores cadastrados, em ordem alfabética."""
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM fornecedor ORDER BY nome")
    dados = cur.fetchall()
    conn.close()
    return dados

def cadastrar_fornecedor(nome: str, cnpj: str, telefone: str = ""):
    """
    Cadastra um novo fornecedor.

    Validações:
    - nome e cnpj são obrigatórios;
    - o CNPJ precisa ser único no banco (restrição UNIQUE definida
      em database/db.py); se já existir, o INSERT falha e o erro é
      repassado para a tela tratar.

    Este cadastro é aberto a qualquer usuário logado (não é
    restrito ao perfil admin), pois cadastrar fornecedor é uma
    atividade operacional do dia a dia do estoque, diferente do
    cadastro de USUÁRIO do sistema, que sim é exclusivo do admin
    por questão de segurança de acesso.
    """
    if not nome or not cnpj:
        raise ValueError("Nome e CNPJ do fornecedor são obrigatórios.")

    conn = get_connection()
    cur = conn.cursor()
    try:
        cur.execute(
            "INSERT INTO fornecedor (nome, cnpj, telefone) VALUES (?, ?, ?)",
            (nome, cnpj, telefone),
        )
        conn.commit()
    finally:
        conn.close()

```


A.5 models/produto.py

```

"""
models/produto.py
=====
Camada de regras de negócio relacionada a PRODUTO e a
MOVIMENTACAO de estoque. É aqui que ficam as validações de dados
e a lógica de "somar/subtrair" a quantidade em estoque a cada
entrada ou saída registrada.
=====
"""

import random
import string

from database.db import get_connection
from models.categoria import obter_ou_criar_categoria

def gerar_codigo_produto(cur) -> str:
    """
    Gera um código de identificação ALEATÓRIO e ÚNICO para um
    novo produto, no formato "PRD-XXXXXX" (6 caracteres
    alfanuméricos maiúsculos após o prefixo, ex.: "PRD-7K2QXB").

    Por que gerar em vez de deixar o usuário digitar?
    - Elimina erro de digitação e códigos "feios"/inconsistentes;
    - Garante, por construção, que nunca existirão dois produtos
      com o mesmo código – requisito pedido pelo usuário.

    Parâmetro:
      cur: cursor já aberto (reutilizamos a mesma conexão/
          transação de quem chamou esta função, em vez de abrir
          uma conexão nova aqui dentro).

    Como garante que é único:
    1) Sorteia um código aleatório usando letras maiúsculas (A-Z)
       e dígitos (0-9);
    2) Consulta o banco para ver se esse código já existe;
    3) Se já existir (chance extremamente baixa, mas não nula),
       sorteia outro e tenta de novo – um laço "while True" que só
       para quando encontra um código livre.

    Por que 6 caracteres alfanuméricos?
    Com 36 símbolos possíveis (26 letras + 10 números) elevado a
    6 posições, temos mais de 2 bilhões de combinações possíveis
    – mais que suficiente para um sistema de controle de estoque
    nunca esgotar códigos disponíveis.
    """
    caracteres = string.ascii_uppercase + string.digits # "A".. "Z" + "0".. "9"
    while True:
        sufixo = "".join(random.choices(caracteres, k=6))
        codigo = f"PRD-{sufixo}"
        cur.execute("SELECT 1 FROM produto WHERE codigo = ?", (codigo,))
        if cur.fetchone() is None: # código livre, pode usar
            return codigo
        # se caiu aqui, o código sorteado já existe – o laço
        # simplesmente tenta de novo com outro sorteio

def validar_numero(valor: str, campo: str) -> float:
    """
    Tenta converter o texto digitado pelo usuário em um número
    (float). Essa função é o coração do requisito "d. Validações
    nos campos para ele não informar texto no lugar de número"
    do enunciado.

    Parâmetros:

```

valor: o texto vindo diretamente de um campo Entry do Tkinter (sempre chega como string, mesmo que o usuário digite números);
 campo: nome do campo, usado só para montar uma mensagem de erro mais clara (ex.: "Quantidade").

Se a conversão falhar (por exemplo, o usuário digitou "abc" em vez de um número), Python lança automaticamente um `TypeError` ou `ValueError`; nós capturamos esse erro e relançamos como uma `ValueError` com mensagem amigável em português, que a tela consegue exibir diretamente em uma caixa de diálogo (messagebox).

```
"""
try:
    return float(valor)
except (TypeError, ValueError):
    raise ValueError(f"O campo '{campo}' deve conter um número válido.")
```

```
def cadastrar_produto(nome, categoria_nome, id_fornecedor, quantidade_inicial,
quantidade_minima, preco_unitario):
```

```
    """
    Cadastra um novo produto no estoque.
```

Parâmetros novos em relação à versão anterior:

```
    categoria_nome: texto digitado/selecionado no combobox de
                    categoria. Pode ser uma categoria já
                    existente ou uma nova – obter_ou_criar_
                    categoria() decide isso automaticamente.
                    Pode vir vazio (categoria é opcional).
    id_fornecedor:  id do fornecedor escolhido no combobox
                    (ou None, se "Nenhum" for selecionado).
                    Diferente da categoria, o fornecedor não
                    é criado "na hora" aqui – ele precisa já
                    existir, cadastrado antes pela tela de
                    Fornecedores (dados como CNPJ exigem um
                    cadastro completo, não só um nome livre).
```

Observação: o parâmetro "codigo" não é recebido de fora – ele é gerado internamente por `gerar_codigo_produto()`, de forma aleatória e garantidamente única, dentro da MESMA transação do INSERT (por isso passamos o mesmo "cur" para as duas operações).

Ordem de validação:

- 1) nome é obrigatório;
- 2) quantidade_inicial, quantidade_minima e preco_unitario passam por `validar_numero()`;
- 3) só depois disso resolvemos a categoria (criando se necessário) e sorteamos o código, e então executamos o INSERT.

Retorno:

O código gerado para o produto (string), para que a tela possa exibi-lo ao usuário logo após o cadastro.

```
"""
if not nome:
    raise ValueError("O nome do produto é obrigatório.")

    quantidade_inicial = int(validar_numero(quantidade_inicial, "Quantidade
inicial"))
    quantidade_minima = int(validar_numero(quantidade_minima, "Quantidade mínima"))
    preco_unitario = validar_numero(preco_unitario, "Preço unitário") # preço pode
ter casas decimais

    conn = get_connection()
    cur = conn.cursor()
    try:
        # obter_ou_criar_categoria devolve o id de uma categoria já
```

```

        # existente com esse nome, ou cria uma nova na hora – tudo
        # dentro desta mesma transação (mesmo "cur").
        id_categoria = obter_ou_criar_categoria(cur, categoria_nome)

        codigo = gerar_codigo_produto(cur) # sorteia um código único, verificando
        contra o banco
        cur.execute(
            """INSERT INTO produto (nome, codigo, id_categoria, id_fornecedor,
quantidade_atual, quantidade_minima, preco_unitario)
            VALUES (?, ?, ?, ?, ?, ?, ?)""",
            (nome, codigo, id_categoria, id_fornecedor, quantidade_inicial,
quantidade_minima, preco_unitario),
        )
        conn.commit()
        return codigo
    finally:
        # Fecha a conexão independentemente do INSERT ter dado
        # certo ou não.
        conn.close()

```

```

def listar_produtos():
    """
    Retorna todos os produtos cadastrados, ordenados por nome, já
    trazendo o NOME da categoria e do fornecedor (não apenas os
    ids), através de dois LEFT JOIN.

    Usamos LEFT JOIN (e não JOIN "normal") porque id_categoria e
    id_fornecedor podem ser NULL – um produto sem fornecedor
    definido, por exemplo, ainda deve aparecer na listagem, só
    que com fornecedor_nome vindo como None.

    Esta função não decide sozinha quais produtos estão "em
    alerta" – ela apenas devolve os dados. É a tela
    (ui/principal.py) quem compara quantidade_atual com
    quantidade_minima e decide se deve destacar a linha em
    vermelho.
    """
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("""
        SELECT
            p.*,
            c.nome AS categoria_nome,
            f.nome AS fornecedor_nome
        FROM produto p
        LEFT JOIN categoria c ON p.id_categoria = c.id_categoria
        LEFT JOIN fornecedor f ON p.id_fornecedor = f.id_fornecedor
        ORDER BY p.nome
    """)
    produtos = cur.fetchall()
    conn.close()
    return produtos

```

```

def registrar_movimentacao(id_produto: int, id_usuario: int, tipo: str, quantidade:
str, observacao: str = ""):
    """
    Registra uma entrada ou saída de estoque para um produto e
    atualiza a quantidade atual desse produto de acordo.

    Parâmetros:
        id_produto: identificador do produto selecionado na lista;
        id_usuario: identificador de quem está logado no sistema
            (para saber "quem" fez a movimentação –
            rastreabilidade);
        tipo: 'entrada' (aumenta o estoque) ou 'saida' (diminui);
        quantidade: texto digitado pelo usuário, ainda não validado;
    """

```

observacao: campo livre, opcional (ex.: "venda", "compra de fornecedor X").

Passo a passo da lógica:

- 1) valida se "tipo" é um dos dois valores aceitos;
- 2) valida se "quantidade" é realmente um número, usando validar_numero(), e garante que seja maior que zero – não faz sentido registrar uma movimentação de 0 ou de valor negativo;
- 3) busca no banco a quantidade atual E o preço unitário atual do produto – o preço é "fotografado" neste exato momento e gravado junto com a movimentação (preco_unitario_momento), preservando o histórico de valores mesmo que o preço do produto seja alterado depois;
- 4) se for uma SAÍDA, verifica se há saldo suficiente – o sistema nunca deixa o estoque ficar negativo;
- 5) calcula a nova quantidade (soma se for entrada, subtrai se for saída);
- 6) grava a nova quantidade na tabela produto E insere um novo registro na tabela movimentacao, dentro da MESMA conexão – isso garante que as duas operações fiquem consistentes entre si (se uma falhasse no meio, nenhuma das duas seria "commitada" de forma parcial, porque só chamamos conn.commit() depois das duas).

Retorno:

A nova quantidade em estoque do produto, já atualizada – útil caso a interface queira exibir uma confirmação com o saldo mais recente.

"""

```
if tipo not in ("entrada", "saida"):
    raise ValueError("Tipo de movimentação inválido.")

quantidade = int(validar_numero(quantidade, "Quantidade"))
if quantidade <= 0:
    raise ValueError("A quantidade deve ser maior que zero.")

conn = get_connection()
cur = conn.cursor()
try:
    # Busca a quantidade atual e o preço unitário vigente ANTES
    # de alterar, para poder calcular o novo saldo, validar
    # saída insuficiente e "fotografar" o preço deste instante.
    cur.execute("SELECT quantidade_atual, preco_unitario FROM produto WHERE
id_produto = ?", (id_produto,))
    row = cur.fetchone()
    if row is None:
        raise ValueError("Produto não encontrado.")

    atual = row["quantidade_atual"]
    preco_momento = row["preco_unitario"]

    # Regra de negócio: não é permitido registrar uma saída
    # maior do que o que existe fisicamente em estoque.
    if tipo == "saida" and quantidade > atual:
        raise ValueError("Quantidade de saída maior que o estoque disponível.")

    # Entrada soma, saída subtrai.
    nova_qtd = atual + quantidade if tipo == "entrada" else atual - quantidade

    # Atualiza o saldo do produto...
    cur.execute("UPDATE produto SET quantidade_atual = ? WHERE id_produto = ?",
(nova_qtd, id_produto))

    # ...e registra o histórico dessa movimentação, incluindo o
    # preço "congelado" no momento (preco_unitario_momento),
    # mantendo rastreabilidade de quem fez o quê, quando e a
    # qual valor (data_hora é preenchida automaticamente pelo
```

```

        # banco).
        cur.execute(
            """INSERT INTO movimentacao (id_produto, id_usuario, tipo, quantidade,
            preco_unitario_momento, observacao)
            VALUES (?, ?, ?, ?, ?, ?)""",
            (id_produto, id_usuario, tipo, quantidade, preco_momento, observacao),
        )

        # Só confirma as duas alterações (UPDATE + INSERT) juntas,
        # no final, quando temos certeza de que tudo deu certo.
        conn.commit()
        return nova_qtd
    finally:
        conn.close()

def listar_movimentacoes():
    """
    Retorna o histórico completo de movimentações (entradas e
    saídas), já com o nome do produto e o nome de quem realizou
    cada movimentação – dados usados pelo relatório de
    movimentações (ver models/relatorio.py).

    Ordenado da mais recente para a mais antiga (DESC), que é a
    ordem mais útil para quem está conferindo o que aconteceu
    recentemente no estoque.
    """
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("""
        SELECT
            m.id_movimentacao,
            p.nome AS produto_nome,
            p.codigo AS produto_codigo,
            u.nome AS usuario_nome,
            m.tipo,
            m.quantidade,
            m.preco_unitario_momento,
            m.data_hora,
            m.observacao
        FROM movimentacao m
        JOIN produto p ON m.id_produto = p.id_produto
        JOIN usuario u ON m.id_usuario = u.id_usuario
        ORDER BY m.data_hora DESC
    """)
    movimentacoes = cur.fetchall()
    conn.close()
    return movimentacoes

```

A.6 models/relatorio.py

```

"""
models/relatorio.py
=====
Geração de relatórios exportáveis do estoque, usando a biblioteca
pandas – exatamente como estudado na Unidade III da disciplina
("Exportação de Arquivos"): o pandas recebe os dados no formato
de lista de dicionários e o DataFrame os organiza automaticamente
em linhas e colunas, prontos para exportar em CSV, Excel ou JSON.

Por que pandas em vez de escrever o CSV "na mão" com o módulo csv?
- Uma única linha de código (to_csv / to_excel) já cuida de
  separadores, cabeçalho e codificação de caracteres;
- O mesmo DataFrame pode ser exportado em vários formatos
  diferentes sem reescrever a lógica de montagem dos dados;
- É a ferramenta que o mercado usa quando o relatório precisa ser
  aberto depois no Excel ou reaproveitado em outra análise.
=====
"""

import pandas as pd

from models.produto import listar_produtos, listar_movimentacoes

def _produtos_para_dataframe() -> pd.DataFrame:
    """
    Converte a listagem de produtos (linhas do SQLite) em um
    DataFrame do pandas.

    Cada linha do banco (sqlite3.Row) é transformada em um
    dicionário {nome_da_coluna: valor}; o pandas.DataFrame recebe
    essa LISTA DE DICIONÁRIOS e monta automaticamente a tabela
    (linhas e colunas), exatamente como descrito na Unidade III.
    """
    produtos = listar_produtos()
    linhas = [dict(p) for p in produtos]
    df = pd.DataFrame(linhas)

    if df.empty:
        return df

    # Coluna calculada: indica visualmente, no próprio relatório,
    # quais produtos estão abaixo da quantidade mínima – a mesma
    # regra usada para o destaque vermelho na tela principal.
    df["situacao"] = df.apply(
        lambda linha: "ABAIXO DO MÍNIMO" if linha["quantidade_atual"] <
        linha["quantidade_minima"] else "OK",
        axis=1,
    )

    # Seleciona e renomeia as colunas para nomes de leitura mais
    # amigável no relatório final (o banco usa nomes técnicos como
    # "categoria_nome"; o relatório usa "Categoria").
    df = df.rename(columns={
        "id_produto": "ID",
        "nome": "Produto",
        "codigo": "Código",
        "categoria_nome": "Categoria",
        "fornecedor_nome": "Fornecedor",
        "quantidade_atual": "Quantidade Atual",
        "quantidade_minima": "Quantidade Mínima",
        "preco_unitario": "Preço Unitário",
        "situacao": "Situação",
    })
    colunas_finais = [
        "ID", "Produto", "Código", "Categoria", "Fornecedor",

```

```

        "Quantidade Atual", "Quantidade Mínima", "Preço Unitário", "Situação",
    ]
    return df[colunas_finais]

def _movimentacoes_para_dataframe() -> pd.DataFrame:
    """
    Mesma ideia da função acima, mas para o histórico de
    movimentações (entradas e saídas).
    """
    movimentacoes = listar_movimentacoes()
    linhas = [dict(m) for m in movimentacoes]
    df = pd.DataFrame(linhas)

    if df.empty:
        return df

    df = df.rename(columns={
        "id_movimentacao": "ID",
        "produto_nome": "Produto",
        "produto_codigo": "Código",
        "usuario_nome": "Usuário",
        "tipo": "Tipo",
        "quantidade": "Quantidade",
        "preco_unitario_momento": "Preço Unitário (no momento)",
        "data_hora": "Data/Hora",
        "observacao": "Observação",
    })
    colunas_finais = [
        "ID", "Data/Hora", "Produto", "Código", "Tipo",
        "Quantidade", "Preço Unitário (no momento)", "Usuário", "Observação",
    ]
    return df[colunas_finais]

def gerar_relatorio_estoque(caminho_arquivo: str):
    """
    Gera o relatório de ESTOQUE ATUAL no formato indicado pela
    extensão do arquivo escolhido pelo usuário:
        .csv -> df.to_csv()
        .xlsx -> df.to_excel() (requer o pacote "openpyxl")

    Levanta ValueError se não houver nenhum produto cadastrado
    (nada para exportar) ou se a extensão do arquivo não for
    suportada.
    """
    df = _produtos_para_dataframe()
    if df.empty:
        raise ValueError("Não há produtos cadastrados para gerar o relatório.")
    _exportar_dataframe(df, caminho_arquivo)

def gerar_relatorio_movimentacoes(caminho_arquivo: str):
    """
    Gera o relatório de HISTÓRICO DE MOVIMENTAÇÕES (entradas e
    saídas), no mesmo esquema de exportação do relatório de
    estoque.
    """
    df = _movimentacoes_para_dataframe()
    if df.empty:
        raise ValueError("Não há movimentações registradas para gerar o
    relatório.")
    _exportar_dataframe(df, caminho_arquivo)

def _exportar_dataframe(df: pd.DataFrame, caminho_arquivo: str):
    """
    Função auxiliar que decide COMO exportar (CSV ou Excel) com

```

```

base na extensão do caminho escolhido pelo usuário na tela.
"""
caminho_lower = caminho_arquivo.lower()

if caminho_lower.endswith(".csv"):
    # index=False evita que o pandas grave uma coluna extra
    # com o número da linha (0, 1, 2...); utf-8-sig garante
    # que acentos apareçam corretos ao abrir o CSV no Excel.
    df.to_csv(caminho_arquivo, index=False, encoding="utf-8-sig", sep=";")
elif caminho_lower.endswith(".xlsx"):
    df.to_excel(caminho_arquivo, index=False, engine="openpyxl")
else:
    raise ValueError("Formato de arquivo não suportado. Escolha .csv
ou .xlsx.")

```


A.7 models/usuario.py

```

"""
models/usuario.py
=====
Esta é a "camada de regras de negócio" (model) relacionada a
USUÁRIO. Aqui ficam as funções que sabem COMO autenticar e
cadastrar usuários – a interface gráfica (pasta ui/) apenas
chama essas funções e mostra o resultado na tela, sem conhecer
detalhes de SQL.

Separar o código dessa forma (banco / regras de negócio /
interface) é uma boa prática chamada "separação de
responsabilidades": se um dia trocarmos o Tkinter por outra
biblioteca de interface, ou o SQLite por outro banco, este
arquivo muda pouco ou nada.
=====
"""

from database.db import get_connection, hash_senha


def autenticar(login: str, senha: str):
    """
    Verifica se o par login/senha corresponde a um usuário
    cadastrado no banco.

    Parâmetros:
        login: texto digitado no campo "Login" da tela;
        senha: texto digitado no campo "Senha" (em texto puro).

    Retorno:
        Uma linha (sqlite3.Row) com os dados do usuário, se as
        credenciais forem válidas; ou None, caso contrário.

    Como funciona a comparação:
    Nunca comparamos a senha digitada diretamente com o que está
    no banco. Em vez disso, aplicamos hash_senha() na senha que
    o usuário acabou de digitar e comparamos o HASH resultante
    com o hash já salvo (senha_hash). Se os hashes forem iguais,
    a senha original também era igual – sem nunca precisarmos
    "descriptografar" nada (SHA-256 não é reversível).
    """
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        "SELECT * FROM usuario WHERE login = ? AND senha_hash = ?",
        (login, hash_senha(senha)),
    )
    usuario = cur.fetchone() # None se não encontrar nenhuma linha correspondente
    conn.close()
    return usuario


def cadastrar_usuario(nome: str, login: str, senha: str, perfil: str):
    """
    Insere um novo usuário no banco de dados.

    Regra de negócio importante: esta função NÃO verifica sozinha
    se quem está chamando é administrador – essa checagem é feita
    na interface (ui/principal.py), que só exibe o botão
    "Cadastrar Usuário" quando o usuário logado tem perfil
    'admin'. Ou seja, a segurança aqui é reforçada em duas camadas:
    1) a tela nem mostra a opção para quem não é admin;
    2) mesmo que alguém tentasse chamar esta função diretamente,
    o campo "perfil" só aceita 'admin' ou 'comum' por causa da
    restrição CHECK definida lá no banco (db.py).
    """

```

```

Validações feitas aqui:
- nome, login e senha não podem estar vazios;
- perfil precisa ser exatamente 'admin' ou 'comum'.
Se alguma validação falhar, é lançada uma ValueError com uma
mensagem amigável, que a tela captura e mostra ao usuário.
"""
if not nome or not login or not senha:
    raise ValueError("Todos os campos são obrigatórios.")
if perfil not in ("admin", "comum"):
    raise ValueError("Perfil inválido.")

conn = get_connection()
cur = conn.cursor()
try:
    # A senha é convertida para hash ANTES de ser gravada –
    # nunca guardamos a senha original no banco.
    cur.execute(
        "INSERT INTO usuario (nome, login, senha_hash, perfil) VALUES (?, ?, ?,
?)",
        (nome, login, hash_senha(senha), perfil),
    )
    conn.commit()
finally:
    # O bloco "finally" garante que a conexão é sempre
    # fechada, mesmo se o INSERT falhar (por exemplo, se o
    # login já existir, já que a coluna é UNIQUE). Nesse caso
    # de erro, a exceção sobe para quem chamou esta função
    # (a tela), que trata isso com uma mensagem ao usuário.
    conn.close()

def listar_usuarios():
    """
    Retorna todos os usuários cadastrados (sem o hash da senha,
    por segurança – só os dados que fazem sentido exibir em uma
    eventual tela de gerenciamento de usuários).
    """
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id_usuario, nome, login, perfil, data_cadastro FROM
usuario")
    dados = cur.fetchall()
    conn.close()
    return dados

```

A.8 ui/login.py

```

"""
ui/login.py
=====
Camada de interface (view) responsável apenas pela TELA DE LOGIN.
Ela não sabe nada sobre SQL ou hash de senha – apenas coleta o
que o usuário digitou e delega a validação para
models/usuario.py (função autenticar), seguindo o princípio de
separar "interface" de "regra de negócio".
=====
"""

import tkinter as tk          # biblioteca gráfica padrão do Python (requisito "g"
do enunciado)
from tkinter import messagebox # sub-módulo do Tkinter para caixas de diálogo
(erro, aviso, sucesso)

from models.usuario import autenticar

class TelaLogin(tk.Tk):
    """
    Esta classe HERDA de tk.Tk, ou seja, ELA PRÓPRIA já é a janela
    principal da aplicação (não precisamos criar uma tk.Tk() à
    parte e depois adicionar widgets nela). Herdar de tk.Tk é um
    padrão comum quando a tela de login é a primeira janela do
    programa.
    """

    def __init__(self, ao_autenticar):
        """
        Parâmetros:
            ao_autenticar: uma função (callback) que será chamada
            automaticamente quando o login der certo, recebendo
            como argumento os dados do usuário autenticado. Quem
            decide o que acontece depois do login (abrir a tela
            principal) é o main.py, não esta classe – isso deixa
            a tela de login reutilizável e desacoplada do resto
            do sistema.
        """
        super().__init__() # inicializa a janela tk.Tk() por trás dos panos
        self.title("Controle de Estoque - Login")
        self.geometry("360x220") # tamanho fixo da janela (largura x altura em
pixels)
        self.resizable(False, False) # impede que o usuário redimensione a janela
de login
        self.ao_autenticar = ao_autenticar

        # --- Título na tela ---
        tk.Label(self, text="Controle de Estoque", font=("Segoe UI", 14,
"bold")).pack(pady=(20, 10))

        # --- Área com os campos de login e senha, organizados em grade (grid) ---
        frame = tk.Frame(self)
        frame.pack(pady=5)

        tk.Label(frame, text="Login:").grid(row=0, column=0, sticky="e", padx=5,
pady=5)
        self.entry_login = tk.Entry(frame)
        self.entry_login.grid(row=0, column=1, padx=5, pady=5)

        tk.Label(frame, text="Senha:").grid(row=1, column=0, sticky="e", padx=5,
pady=5)
        # show="*" faz com que os caracteres digitados apareçam
        # como asteriscos na tela, escondendo a senha de quem
        # está olhando por cima do ombro.
        self.entry_senha = tk.Entry(frame, show="*")

```

```

self.entry_senha.grid(row=1, column=1, padx=5, pady=5)

# --- Botão de ação ---
tk.Button(self, text="Entrar", width=15,
command=self.efetuar_login).pack(pady=15)
tk.Label(self, text="Usuário padrão: admin / admin123", fg="gray").pack()

# Permite pressionar a tecla Enter em qualquer lugar da
# janela para efetuar o login, sem precisar clicar no
# botão – pequeno detalhe de usabilidade (requisito "e").
self.bind("<Return>", lambda event: self.efetuar_login())

# Já deixa o cursor piscando no campo de login ao abrir a tela.
self.entry_login.focus()

def efetuar_login(self):
    """
    Chamada quando o usuário clica em "Entrar" ou pressiona Enter.

    Fluxo:
    1) Lê o texto dos campos de login e senha;
    2) Se algum estiver vazio, mostra um aviso e para aqui
       (validação de campo obrigatório, requisito "d");
    3) Chama autenticar(login, senha) do model – que faz o
       hash da senha e consulta o banco;
    4) Se autenticar() devolver None, mostra mensagem de erro
       e mantém a tela de login aberta;
    5) Se dermos certo, fechamos esta janela (self.destroy())
       e chamamos o callback ao_autenticar, passando os dados
       do usuário logado – é esse callback (definido em
       main.py) que abre a tela principal.
    """
    login = self.entry_login.get().strip() # .strip() remove espaços
    # acidentais no início/fim
    senha = self.entry_senha.get()

    if not login or not senha:
        messagebox.showwarning("Campos obrigatórios", "Informe login e senha.")
        return

    usuario = autenticar(login, senha)
    if usuario is None:
        messagebox.showerror("Erro de autenticação", "Login ou senha
        inválidos.")
        return

    self.destroy() # fecha a janela de login
    self.ao_autenticar(usuario) # "entrega" o usuário autenticado para quem
    chamou esta tela

```

A.9 ui/principal.py

```

"""
ui/principal.py
=====
Camada de interface (view) responsável pela TELA PRINCIPAL da
aplicação: listagem de estoque, cadastro de produto (agora com
categoria e fornecedor relacionados), registro de movimentações
(entrada/saída, já guardando o preço histórico), cadastro de
fornecedores e, quando o usuário é admin, cadastro de novos
usuários.

Assim como em login.py, esta tela apenas coleta dados e delega
toda a lógica de validação/gravação para os models.
=====
"""

import tkinter as tk
from tkinter import ttk, messagebox, filedialog # filedialog abre a janela nativa
"Salvar como..."

from models.produto import cadastrar_produto, listar_produtos,
registrar_movimentacao
from models.usuario import cadastrar_usuario
from models.categoria import listar_categorias
from models.fornecedor import listar_fornecedores, cadastrar_fornecedor
from models.relatorio import gerar_relatorio_estoque, gerar_relatorio_movimentacoes

class TelaPrincipal(tk.Tk):
    def __init__(self, usuario):
        """
        Parâmetro:
            usuario: a linha (sqlite3.Row) do usuário autenticado,
            recebida da tela de login. Guardamos essa referência
            em self.usuario porque ela é usada em vários pontos:
            - para exibir o nome/perfil no título da janela;
            - para saber se deve ou não mostrar o botão de
              "Cadastrar Usuário" (somente perfil admin);
            - para registrar QUEM fez cada movimentação de
              estoque (rastreadabilidade – requisito "j. Sessão de
              usuário").
        """
        super().__init__()
        self.usuario = usuario
        self.title(f"Controle de Estoque – {usuario['nome']}
        ({usuario['perfil']})")
        self.geometry("900x480")

        self._montar_menu() # monta a barra de botões no topo
        self._montar_lista() # monta a tabela de produtos
        self.atualizar_lista() # já carrega os dados do banco assim que a tela
        abre

        # =====
        # MENU / BARRA DE BOTÕES
        # =====
        def _montar_menu(self):
            barra = tk.Frame(self, pady=8)
            barra.pack(fill="x")

            tk.Button(barra, text="Novo Produto",
            command=self.abrir_cadastro_produto).pack(side="left", padx=5)
            tk.Button(barra, text="Novo Fornecedor",
            command=self.abrir_cadastro_fornecedor).pack(side="left", padx=5)
            tk.Button(barra, text="Registrar Entrada", command=lambda:
            self.abrir_movimentacao("entrada")).pack(side="left", padx=5)

```

```

        tk.Button(barra, text="Registrar Saída", command=lambda:
self.abrir_movimentacao("saida")).pack(side="left", padx=5)
        tk.Button(barra, text="Atualizar Lista",
command=self.atualizar_lista).pack(side="left", padx=5)
        tk.Button(barra, text="Gerar Relatório",
command=self.abrir gerar_relatorio).pack(side="left", padx=5)

        # Este botão SÓ é criado se o perfil do usuário logado for
        # 'admin' – requisito "Somente administrador poderá
        # cadastrar usuário".
        if self.usuario["perfil"] == "admin":
            tk.Button(barra, text="Cadastrar Usuário",
command=self.abrir_cadastro_usuario).pack(side="left", padx=5)

# =====
# LISTAGEM (TABELA DE PRODUTOS)
# =====
def _montar_lista(self):
    """
    Cria a tabela (Treeview) que exibe os produtos, agora
    incluindo as colunas "Categoria" e "Fornecedor" (antes só
    existia "Categoria" como texto livre; agora ambas vêm dos
    LEFT JOIN feitos em models/produto.py -> listar_produtos()).
    """
    colunas = ("id", "nome", "codigo", "categoria", "fornecedor", "quantidade",
"minimo", "preco")
    self.tree = ttk.Treeview(self, columns=colunas, show="headings")

    titulos = {
        "id": "ID", "nome": "Nome", "codigo": "Código", "categoria":
"Categoria",
        "fornecedor": "Fornecedor", "quantidade": "Qtd. Atual", "minimo": "Qtd.
Mínima",
        "preco": "Preço Unit.",
    }
    larguras = {"nome": 130, "fornecedor": 130, "categoria": 100}
    for c in colunas:
        self.tree.heading(c, text=titulos[c])
        self.tree.column(c, width=larguras.get(c, 90), anchor="center")

    # Tag de destaque visual para produtos com estoque abaixo
    # do mínimo (requisito de alerta).
    self.tree.tag_configure("alerta", background="#f8d7da")

    self.tree.pack(fill="both", expand=True, padx=10, pady=10)

def atualizar_lista(self):
    """
    Recarrega a tabela com os dados mais recentes do banco,
    incluindo o nome da categoria e do fornecedor (que agora
    vêm de tabelas relacionadas, não mais de texto livre).
    """
    for item in self.tree.get_children():
        self.tree.delete(item)

    for p in listar_produtos():
        tag = "alerta" if p["quantidade_atual"] < p["quantidade_minima"] else
""
        self.tree.insert("", "end", values=(
            p["id_produto"], p["nome"], p["codigo"],
            p["categoria_nome"] or "-", p["fornecedor_nome"] or "-",
            p["quantidade_atual"], p["quantidade_minima"], f"R$
{p['preco_unitario']:.2f}",
        ), tags=(tag,))

def _produto_selecionado(self):
    selecao = self.tree.selection()
    if not selecao:

```

```

        return None
    return self.tree.item(selecao[0])["values"]

# =====
# JANELA: CADASTRO DE PRODUTO
# =====
def abrir_cadastro_produto(self):
    """
    Formulário de cadastro de produto, agora com:
    - Combobox EDITÁVEL de categoria: o usuário pode escolher
      uma categoria já existente na lista, OU digitar um nome
      novo – nesse caso, models/categoria.py cria a categoria
      automaticamente ao salvar (função obter_ou_criar_categoria);
    - Combobox SOMENTE LEITURA de fornecedor: como fornecedor
      exige dados completos (CNPJ), ele precisa ser cadastrado
      antes, pelo botão "Novo Fornecedor"; aqui só é possível
      selecionar um já existente (ou deixar "Nenhum").
    """
    janela = tk.Toplevel(self)
    janela.title("Novo Produto")
    janela.geometry("340x380")
    janela.grab_set()

    tk.Label(
        janela, text="O código do produto será gerado automaticamente ao
salvar.",
        fg="gray", justify="center",
    ).grid(row=0, column=0, columnspan=2, pady=(10, 5))

    campos = {}
    rotulos_texto = [
        ("nome", "Nome*"), ("quantidade_inicial", "Qtd. Inicial*"),
        ("quantidade_minima", "Qtd. Mínima*"), ("preco_unitario", "Preço
Unitário*"),
    ]

    # --- Campo Categoria (Combobox editável) ---
    tk.Label(janela, text="Categoria").grid(row=1, column=0, sticky="e",
padx=5,
pady=5)
    nomes_categorias = [c["nome"] for c in listar_categorias()]
    combo_categoria = ttk.Combobox(janela, values=nomes_categorias) # editável
por padrão (sem state="readonly")
    combo_categoria.grid(row=1, column=1, padx=5, pady=5)

    # --- Campo Fornecedor (Combobox somente leitura) ---
    tk.Label(janela, text="Fornecedor").grid(row=2, column=0, sticky="e",
padx=5,
pady=5)
    fornecedores = listar_fornecedores()
    # Guardamos os ids em uma lista paralela na MESMA ordem dos
    # textos exibidos, para conseguir "traduzir" de volta o
    # texto escolhido para o id_fornecedor correspondente.
    opcoes_fornecedor = ["-- Nenhum --"] + [f"{f['nome']} ({f['cnpj']})" for f
in fornecedores]
    ids_fornecedor = [None] + [f["id_fornecedor"] for f in fornecedores]
    combo_fornecedor = ttk.Combobox(janela, values=opcoes_fornecedor,
state="readonly")
    combo_fornecedor.current(0) # começa em "-- Nenhum --"
    combo_fornecedor.grid(row=2, column=1, padx=5, pady=5)

    # --- Demais campos numéricos/texto, gerados dinamicamente ---
    for i, (chave, rotulo) in enumerate(rotulos_texto, start=3):
        tk.Label(janela, text=rotulo).grid(row=i, column=0, sticky="e", padx=5,
pady=5)
        entrada = tk.Entry(janela)
        entrada.grid(row=i, column=1, padx=5, pady=5)
        campos[chave] = entrada

    def salvar():

```

```

try:
    # Traduz o texto escolhido no combobox de fornecedor
    # de volta para o id_fornecedor correspondente (ou
    # None, se "-- Nenhum --" estiver selecionado).
    indice = combo_fornecedor.current()
    id_fornecedor = ids_fornecedor[indice] if indice >= 0 else None

    codigo_gerado = cadastrar_produto(
        campos["nome"].get().strip(),
        combo_categoria.get().strip(),
        id_fornecedor,
        campos["quantidade_inicial"].get(),
        campos["quantidade_minima"].get(),
        campos["preco_unitario"].get(),
    )
    messagebox.showinfo("Sucesso", f"Produto cadastrado com sucesso!\n
nCódigo gerado: {codigo_gerado}")
    janela.destroy()
    self.atualizar_lista()
except ValueError as erro:
    messagebox.showerror("Erro de validação", str(erro))
except Exception:
    messagebox.showerror("Erro", "Código de produto já existente ou
dado inválido.")

tk.Button(janela, text="Salvar", command=salvar).grid(
    row=3 + len(rotulos_texto), column=0, columnspan=2, pady=15
)

# =====
# JANELA: CADASTRO DE FORNECEDOR (NOVA)
# =====
def abrir_cadastro_fornecedor(self):
    """
    Cadastro de fornecedor, aberto a qualquer usuário logado
    (não é restrito ao admin – diferente do cadastro de
    usuário do sistema). Depois de cadastrado, o fornecedor
    passa a aparecer no combobox de "Novo Produto".
    """
    janela = tk.Toplevel(self)
    janela.title("Novo Fornecedor")
    janela.geometry("300x220")
    janela.grab_set()

    campos = {}
    for i, (chave, rotulo) in enumerate([("nome", "Nome*"), ("cnpj", "CNPJ*"),
    ("telefone", "Telefone")]):
        tk.Label(janela, text=rotulo).grid(row=i, column=0, sticky="e", padx=5,
pady=5)
        entrada = tk.Entry(janela)
        entrada.grid(row=i, column=1, padx=5, pady=5)
        campos[chave] = entrada

    def salvar():
        try:
            cadastrar_fornecedor(
                campos["nome"].get().strip(),
                campos["cnpj"].get().strip(),
                campos["telefone"].get().strip(),
            )
            messagebox.showinfo("Sucesso", "Fornecedor cadastrado com
sucesso.")
            janela.destroy()
        except ValueError as erro:
            messagebox.showerror("Erro de validação", str(erro))
        except Exception:
            # Erro mais comum aqui: CNPJ duplicado (coluna UNIQUE).

```



```

        messagebox.showerror("Erro", "CNPJ já cadastrado para outro
fornecedor.")

        tk.Button(janela, text="Salvar", command=salvar).grid(row=3, column=0,
columnspan=2, pady=15)

# =====
# JANELA: REGISTRAR ENTRADA / SAÍDA (MOVIMENTAÇÃO)
# =====
def abrir_movimentacao(self, tipo):
    """
    O preço unitário do produto é capturado automaticamente
    pelo model (registrar_movimentacao), sem precisar de mais
    nenhum campo aqui na tela – ele "fotografa" o preço atual
    do produto no instante da movimentação.
    """
    produto = self._produto_selecionado()
    if not produto:
        messagebox.showwarning("Seleção necessária", "Selecione um produto na
lista.")
    return

    janela = tk.Toplevel(self)
    titulo = "Registrar Entrada" if tipo == "entrada" else "Registrar Saída"
    janela.title(titulo)
    janela.geometry("300x180")
    janela.grab_set()

    # produto[1] é o "nome" do produto (colunas definidas em
    # _montar_lista: id=0, nome=1, codigo=2, categoria=3, ...).
    tk.Label(janela, text=f"Produto: {produto[1]}", font=("Segoe UI", 10,
"bold")).pack(pady=(15, 5))
    tk.Label(janela, text="Quantidade*").pack()
    entrada_qtd = tk.Entry(janela)
    entrada_qtd.pack(pady=5)
    tk.Label(janela, text="Observação").pack()
    entrada_obs = tk.Entry(janela)
    entrada_obs.pack(pady=5)

    def salvar():
        try:
            registrar_movimentacao(
                produto[0], self.usuario["id_usuario"], tipo,
                entrada_qtd.get(), entrada_obs.get().strip(),
            )
            messagebox.showinfo("Sucesso", "Movimentação registrada com
sucesso.")
            janela.destroy()
            self.atualizar_lista()
        except ValueError as erro:
            messagebox.showerror("Erro de validação", str(erro))

    tk.Button(janela, text="Confirmar", command=salvar).pack(pady=10)

# =====
# JANELA: GERAR RELATÓRIO (NOVA)
# =====
def abrir gerar_relatorio(self):
    """
    Permite ao usuário escolher QUAL relatório exportar
    (estoque atual ou histórico de movimentações) e em QUAL
    formato (CSV ou Excel), usando pandas nos bastidores
    (models/relatorio.py) – conteúdo estudado na Unidade III
    da disciplina ("Exportação de Arquivos").

    Fluxo:
    1) o usuário escolhe o tipo de relatório em um combobox;
    2) ao clicar em "Gerar", abrimos a janela NATIVA do sistema

```

```

operacional "Salvar como..." (filedialog), para que o
usuário escolha ONDE salvar e o NOME do arquivo;
3) chamamos a função correspondente do model, que decide
entre CSV/Excel de acordo com a extensão escolhida.
"""
janela = tk.Toplevel(self)
janela.title("Gerar Relatório")
janela.geometry("320x200")
janela.grab_set()

tk.Label(janela, text="Tipo de relatório").grid(row=0, column=0,
sticky="e", padx=5, pady=(20, 5))
tipo_var = tk.StringVar(value="Estoque atual")
ttk.Combobox(
    janela, textvariable=tipo_var,
    values=["Estoque atual", "Histórico de movimentações"],
    state="readonly", width=22,
).grid(row=0, column=1, padx=5, pady=(20, 5))

tk.Label(janela, text="Formato do arquivo").grid(row=1, column=0,
sticky="e", padx=5, pady=5)
formato_var = tk.StringVar(value="CSV (.csv)")
ttk.Combobox(
    janela, textvariable=formato_var,
    values=["CSV (.csv)", "Excel (.xlsx)"],
    state="readonly", width=22,
).grid(row=1, column=1, padx=5, pady=5)

def gerar():
    # Traduz a escolha do combobox de formato para a
    # extensão e o filtro que o filedialog deve usar.
    if formato_var.get().startswith("CSV"):
        extensao, filtro = ".csv", [("Arquivo CSV", "*.csv")]
    else:
        extensao, filtro = ".xlsx", [("Arquivo Excel", "*.xlsx")]

    nome_sugerido = (
        "relatorio_estoque" if tipo_var.get() == "Estoque atual"
        else "relatorio_movimentacoes"
    )

    # Abre a janela nativa "Salvar como..." do sistema
    # operacional (funciona igual no Pop!_OS/Linux, Windows
    # e macOS), já sugerindo nome e extensão do arquivo.
    caminho = filedialog.asksaveasfilename(
        parent=janela,
        title="Salvar relatório como...",
        initialfile=nome_sugerido + extensao,
        defaultextension=extensao,
        filetypes=filtro,
    )
    if not caminho: # usuário clicou em "Cancelar"
        return

    try:
        if tipo_var.get() == "Estoque atual":
            gerar_relatorio_estoque(caminho)
        else:
            gerar_relatorio_movimentacoes(caminho)
        messagebox.showinfo("Sucesso", f"Relatório gerado com sucesso em:\n{caminho}")
        janela.destroy()
    except ValueError as erro:
        messagebox.showerror("Erro ao gerar relatório", str(erro))
    except Exception as erro:
        messagebox.showerror("Erro ao gerar relatório", f"Não foi possível salvar o arquivo.\n{erro}")

```

```

        tk.Button(janela, text="Gerar", command=gerar).grid(row=2, column=0,
columnspan=2, pady=20)

# =====
# JANELA: CADASTRO DE USUÁRIO (SOMENTE ADMINISTRADOR)
# =====
def abrir_cadastro_usuario(self):
    janela = tk.Toplevel(self)
    janela.title("Novo Usuário")
    janela.geometry("300x260")
    janela.grab_set()

    campos = {}
    for i, (chave, rotulo) in enumerate([("nome", "Nome*"), ("login",
"Login*"), ("senha", "Senha*")]):
        tk.Label(janela, text=rotulo).grid(row=i, column=0, sticky="e", padx=5,
pady=5)
        entrada = tk.Entry(janela, show="*" if chave == "senha" else None)
        entrada.grid(row=i, column=1, padx=5, pady=5)
        campos[chave] = entrada

    tk.Label(janela, text="Perfil*").grid(row=3, column=0, sticky="e", padx=5,
pady=5)
    perfil_var = tk.StringVar(value="comum")
    tk.OptionMenu(janela, perfil_var, "comum", "admin").grid(row=3, column=1,
padx=5, pady=5)

    def salvar():
        try:
            cadastrar_usuario(
                campos["nome"].get().strip(), campos["login"].get().strip(),
                campos["senha"].get(), perfil_var.get(),
            )
            messagebox.showinfo("Sucesso", "Usuário cadastrado com sucesso.")
            janela.destroy()
        except ValueError as erro:
            messagebox.showerror("Erro de validação", str(erro))
        except Exception:
            messagebox.showerror("Erro", "Login já existente.")

    tk.Button(janela, text="Salvar", command=salvar).grid(row=4, column=0,
columnspan=2, pady=15)

```

A.10 webapp.py

```

"""
webapp.py
=====
Interface WEB da aplicação de Controle de Estoque, construída com
Flask – a segunda opção de biblioteca de interface prevista no
enunciado ("g. ... tkinter / flask").

PONTO CENTRAL DESTA ARQUIVO: ele NÃO duplica nenhuma regra de
negócio. Todas as validações, todo o acesso ao banco de dados e
toda a lógica de cadastro/movimentação continuam morando
exclusivamente em models/ (os mesmos arquivos usados por
ui/principal.py no Tkinter). Este arquivo só faz o trabalho de
"tradutor": recebe requisições HTTP, chama as funções de
models/, e devolve páginas HTML.

Por que isso é importante para a sincronização desktop <-> web?
Como as duas interfaces (Tkinter e Flask) chamam as MESMAS
funções de models/, que por sua vez sempre abrem uma conexão
NOVA com o arquivo database/estoque.db a cada operação (veja
get_connection() em database/db.py), qualquer alteração feita em
uma interface fica gravada no arquivo .db imediatamente – e a
outra interface enxerga essa mudança na próxima vez que consultar
o banco (no Flask, isso acontece a cada requisição/recarregamento
de página; no Tkinter, ao clicar em "Atualizar Lista"). Não há
cache nem cópia de dados em memória entre as duas interfaces.
=====
"""

import os
import tempfile

from flask import Flask, render_template, request, redirect, url_for, session,
flash, send_file

from database.db import criar_tabelas
from models.usuario import autenticar, cadastrar_usuario
from models.categoria import listar_categorias
from models.fornecedor import listar_fornecedores, cadastrar_fornecedor
from models.produto import cadastrar_produto, listar_produtos,
registrar_movimentacao, listar_movimentacoes
from models.relatorio import gerar_relatorio_estoque, gerar_relatorio_movimentacoes

app = Flask(__name__)
# chave usada pelo Flask para assinar o cookie de sessão (login);
# em um ambiente de produção real isso viria de uma variável de
# ambiente, mas para um projeto acadêmico local um valor fixo é
# suficiente.
app.secret_key = "chave-secreta-controle-estoque-unifecaf"

# -----
# Pequeno "guardião" de autenticação: as rotas que exigem login
# chamam esta função no início; se não houver usuário na sessão,
# redireciona para a tela de login. Isso evita repetir a mesma
# checagem em cada rota.
# -----
def usuario_logado():
    return session.get("usuario_id") is not None

# =====
# LOGIN / LOGOUT
# =====
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":

```

```

login_digitado = request.form.get("login", "").strip()
senha_digitada = request.form.get("senha", "")

if not login_digitado or not senha_digitada:
    flash("Informe login e senha.", "erro")
    return render_template("login.html")

usuario = autenticar(login_digitado, senha_digitada)
if usuario is None:
    flash("Login ou senha inválidos.", "erro")
    return render_template("login.html")

# Guarda os dados do usuário na sessão (equivalente ao
# "self.usuario" que a TelaPrincipal do Tkinter guarda em
# memória) – é assim que o Flask mantém a "sessão de
# usuário" exigida pelo enunciado, item "j".
session["usuario_id"] = usuario["id_usuario"]
session["usuario_nome"] = usuario["nome"]
session["perfil"] = usuario["perfil"]
return redirect(url_for("dashboard"))

return render_template("login.html")

@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("login"))

# =====
# DASHBOARD (equivalente à TelaPrincipal do Tkinter)
# =====
@app.route("/")
def dashboard():
    if not usuario_logado():
        return redirect(url_for("login"))

    # listar_produtos() é chamada de zero a cada requisição – é
    # justamente isso que garante que, se um produto foi
    # cadastrado ou movimentado pelo Tkinter um segundo atrás,
    # ele já aparece aqui ao recarregar a página.
    produtos = listar_produtos()
    return render_template("dashboard.html", produtos=produtos)

# =====
# PRODUTOS
# =====
@app.route("/produtos/novo", methods=["GET", "POST"])
def novo_produto():
    if not usuario_logado():
        return redirect(url_for("login"))

    if request.method == "POST":
        try:
            id_fornecedor = request.form.get("id_fornecedor") or None
            codigo = cadastrar_produto(
                request.form.get("nome", "").strip(),
                request.form.get("categoria", "").strip(),
                int(id_fornecedor) if id_fornecedor else None,
                request.form.get("quantidade_inicial", ""),
                request.form.get("quantidade_minima", ""),
                request.form.get("preco_unitario", ""),
            )
            flash(f"Produto cadastrado com sucesso! Código gerado: {codigo}",
                  "sucesso")
            return redirect(url_for("dashboard"))

```

```

        except ValueError as erro:
            flash(str(erro), "erro")
        except Exception:
            flash("Código de produto já existente ou dado inválido.", "erro")

    return render_template(
        "produto_form.html",
        categorias=listar_categorias(),
        fornecedores=listar_fornecedores(),
    )

# =====
# FORNECEDORES
# =====
@app.route("/fornecedores/novo", methods=["GET", "POST"])
def novo_fornecedor():
    if not usuario_logado():
        return redirect(url_for("login"))

    if request.method == "POST":
        try:
            cadastrar_fornecedor(
                request.form.get("nome", "").strip(),
                request.form.get("cnpj", "").strip(),
                request.form.get("telefone", "").strip(),
            )
            flash("Fornecedor cadastrado com sucesso.", "sucesso")
            return redirect(url_for("dashboard"))
        except ValueError as erro:
            flash(str(erro), "erro")
        except Exception:
            flash("CNPJ já cadastrado para outro fornecedor.", "erro")

    return render_template("fornecedor_form.html")

# =====
# MOVIMENTAÇÃO (entrada/saída)
# =====
@app.route("/movimentacao/<int:id_produto>/<tipo>", methods=["GET", "POST"])
def movimentacao(id_produto, tipo):
    if not usuario_logado():
        return redirect(url_for("login"))
    if tipo not in ("entrada", "saida"):
        return redirect(url_for("dashboard"))

    if request.method == "POST":
        try:
            registrar_movimentacao(
                id_produto, session["usuario_id"], tipo,
                request.form.get("quantidade", ""),
                request.form.get("observacao", "").strip(),
            )
            flash("Movimentação registrada com sucesso.", "sucesso")
            return redirect(url_for("dashboard"))
        except ValueError as erro:
            flash(str(erro), "erro")

    produto = next((p for p in listar_produtos() if p["id_produto"] == id_produto),
None)
    if produto is None:
        return redirect(url_for("dashboard"))
    return render_template("movimentacao_form.html", produto=produto, tipo=tipo)

# =====
# USUÁRIOS (somente admin – mesma regra do Tkinter)

```

```

# =====
@app.route("/usuarios/novo", methods=["GET", "POST"])
def novo_usuario():
    if not usuario_logado():
        return redirect(url_for("login"))
    if session.get("perfil") != "admin":
        flash("Apenas administradores podem cadastrar usuários.", "erro")
        return redirect(url_for("dashboard"))

    if request.method == "POST":
        try:
            cadastrar_usuario(
                request.form.get("nome", "").strip(),
                request.form.get("login", "").strip(),
                request.form.get("senha", ""),
                request.form.get("perfil", "comum"),
            )
            flash("Usuário cadastrado com sucesso.", "sucesso")
            return redirect(url_for("dashboard"))
        except ValueError as erro:
            flash(str(erro), "erro")
        except Exception:
            flash("Login já existente.", "erro")

    return render_template("usuario_form.html")

# =====
# RELATÓRIOS (reaproveita models/relatorio.py, o mesmo do Tkinter)
# =====
@app.route("/relatorios")
def relatorios():
    if not usuario_logado():
        return redirect(url_for("login"))
    return render_template("relatorios.html")

@app.route("/relatorios/baixar/<tipo>/<formato>")
def baixar_relatorio(tipo, formato):
    if not usuario_logado():
        return redirect(url_for("login"))

    extensao = ".csv" if formato == "csv" else ".xlsx"
    # Gera o arquivo em uma pasta temporária do sistema e o envia
    # como download – mesma função usada pelo botão "Gerar
    # Relatório" do Tkinter, só que aqui o "Salvar como..." é o
    # próprio navegador que cuida disso.
    caminho_temporario = os.path.join(tempfile.gettempdir(), f"relatorio_{tipo}{extensao}")
    try:
        if tipo == "estoque":
            gerar_relatorio_estoque(caminho_temporario)
        else:
            gerar_relatorio_movimentacoes(caminho_temporario)
        return send_file(caminho_temporario, as_attachment=True)
    except ValueError as erro:
        flash(str(erro), "erro")
        return redirect(url_for("relatorios"))

if __name__ == "__main__":
    criar_tabelas() # garante que o banco/tabelas existem antes de aceitar
    requisições
    # host="0.0.0.0" permite acessar de outros dispositivos na
    # mesma rede (ex.: celular), não só de "localhost". debug=True
    # só deve ser usado em desenvolvimento (nunca em produção).
    app.run(host="0.0.0.0", port=5000, debug=True, use_reloader=False)

```


A.11 templates/base.html

```

<!--
  templates/base.html
  =====
  Template BASE (layout-mãe) da interface web, construído com o
  motor de templates Jinja2 (embutido no Flask). Todos os outros
  templates (login.html, dashboard.html, produto_form.html etc.)
  "herdam" este arquivo por meio da tag Jinja "extends" e só
  precisam preencher o bloco de conteúdo próprio de cada tela
  (ver o marcador logo abaixo do <main>, no fim deste arquivo).

  Essa herança de template é o equivalente, na camada web, ao que
  a classe TelaPrincipal(tk.Tk) faz no Tkinter: definir uma vez
  a "moldura" comum (cabeçalho, menu, largura, cores) para que
  cada tela específica só cuide do seu próprio conteúdo.
  =====
-->
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Controle de Estoque{% if titulo %} – {{ titulo }}{% endif %}</title>
  <style>
    /* =====
    FOLHA DE ESTILOS (CSS) – comentada seção por seção.
    Não há dependência de nenhum framework externo (Bootstrap,
    Tailwind etc.) nem de conexão com a internet: todo o
    visual é definido aqui mesmo, em CSS puro, o que mantém a
    aplicação 100% funcional mesmo sem acesso à rede – um
    requisito implícito de robustez para um sistema de
    controle de estoque, que pode rodar em ambientes com
    conectividade instável.
    ===== */

    /* ----- Reset básico e tipografia geral ----- */
    /* Remove a margem padrão do <body> (todo navegador aplica uma
    margem de ~8px por padrão) e define uma fonte de sistema
    (Segoe UI no Windows, e as demais como alternativa em
    outros sistemas operacionais), garantindo boa legibilidade
    sem precisar carregar uma fonte externa (Google Fonts). */
    body {
      font-family: 'Segoe UI', Arial, sans-serif;
      margin: 0;
      background: #eef1f4;          /* cinza-azulado bem claro, evita o
branco "cru" */
      color: #222;
    }

    /* ----- Cabeçalho fixo no topo ----- */
    /* display:flex + justify-content:space-between distribui o
    título à esquerda e a navegação (usuário logado, links) à
    direita, sem precisar de posicionamento manual (position:
    absolute), o que também deixa o cabeçalho responsivo caso
    a janela do navegador seja redimensionada. */
    header {
      background: linear-gradient(90deg, #1f2f3d, #2c3e50);
      color: white;
      padding: 16px 28px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      box-shadow: 0 2px 6px rgba(0,0,0,0.2);  /* dá profundidade, "descola"
o cabeçalho do conteúdo */
    }
    /* Título do sistema, com um pequeno "selo" (ícone via emoji de
    texto, sem depender de biblioteca de ícones) para reforçar
    a identidade visual nos prints/capturas de tela do relatório. */

```

```

header .marca {
    font-size: 19px;
    font-weight: 600;
    letter-spacing: 0.3px;
}
header a { color: white; text-decoration: none; margin-left: 14px; }
header nav { display: flex; align-items: center; gap: 6px; }
header nav span { opacity: 0.85; font-size: 13px; margin-right: 10px; }
/* Os links de navegação (Início/Relatórios/Sair) recebem um
fundo próprio para parecerem "botões", diferenciando-os do
texto solto do cabeçalho, e uma transição suave de cor no
hover – um detalhe pequeno, mas que comunica visualmente
que o elemento é clicável. */
header nav a {
    background: rgba(255,255,255,0.08);
    padding: 7px 14px;
    border-radius: 5px;
    font-size: 13px;
    transition: background 0.15s ease-in-out;
}
header nav a:hover { background: rgba(255,255,255,0.2); }

/* ----- Área de conteúdo principal ----- */
/* max-width limita a largura do cartão de conteúdo em telas
grandes (evita textos/tabelas esticados demais em monitores
largos), enquanto "margin: 24px auto" centraliza esse
cartão horizontalmente. */
main {
    max-width: 960px;
    margin: 28px auto;
    background: white;
    padding: 28px 32px;
    border-radius: 8px;
    box-shadow: 0 1px 4px rgba(0,0,0,0.12);
}
main h2 { margin-top: 0; color: #1f2f3d; }
main h3 { color: #1f2f3d; margin-bottom: 6px; }

/* ----- Tabela de listagem (dashboard) ----- */
table { border-collapse: collapse; width: 100%; margin-top: 12px; }
th, td { border: 1px solid #e2e6ea; padding: 10px 12px; text-align: center;
font-size: 14px; }
th {
    background: #f4f6f8;
    color: #1f2f3d;
    text-transform: uppercase; /* cabeçalhos em caixa alta, padrão comum
de dashboards */
    font-size: 12px;
    letter-spacing: 0.4px;
}
/* Efeito "hover": destaca a linha sob o cursor do mouse,
facilitando a leitura de tabelas com várias colunas – o
usuário não perde a linha de vista ao mover os olhos da
esquerda para a direita. Só se aplica a linhas SEM a
classe "alerta", para não competir visualmente com o
destaque vermelho de estoque baixo (regra de negócio tem
prioridade sobre o efeito estético). */
tbody tr:hover:not(.alerta) { background: #f8fafc; }
/* Alerta de estoque abaixo do mínimo – mesma cor de fundo já
usada nas mensagens de erro (.flash.erro), reforçando por
repetição visual que "vermelho = atenção" em todo o sistema. */
tr.alerta { background: #f8d7da; font-weight: 600; }
tr.alerta:hover { background: #f5c6cb; }

/* ----- Mensagens de feedback (flash messages) ----- */
.flash { padding: 12px 16px; border-radius: 6px; margin-bottom: 16px; font-
size: 14px; }

```

```

        .flash.sucesso { background: #d4edda; color: #155724; border-left: 4px
solid #28a745; }
        .flash.erro { background: #f8d7da; color: #721c24; border-left: 4px solid
#dc3545; }

        /* ----- Formulários ----- */
        /* Os seletores abaixo padronizam o espaçamento vertical entre
campos (margin-top no label "empurra" cada grupo
label+input para longe do campo anterior), e limitam a
largura máxima dos campos de texto (max-width) para que
não se estiquem por toda a largura do <main> em telas
largas – formulários muito largos são mais difíceis de
escanear visualmente. */
        label { display: block; margin-top: 16px; margin-bottom: 4px; font-weight:
600; font-size: 13px; color: #444; }
        input, select {
            padding: 9px 10px;
            width: 100%;
            max-width: 340px;
            margin-top: 2px;
            box-sizing: border-box;          /* garante que padding não estoure a
largura definida */
            border: 1px solid #ccd3da;
            border-radius: 5px;
            font-size: 14px;
        }
        input:focus, select:focus {
            outline: none;
            border-color: #2c3e50;
            box-shadow: 0 0 2px rgba(44,62,80,0.15); /* anel de foco discreto,
sem depender do estilo padrão do navegador */
        }

        /* ----- Botões ----- */
        button, .botao {
            background: #2c3e50;
            color: white;
            border: none;
            padding: 10px 18px;
            border-radius: 5px;
            cursor: pointer;
            margin-top: 18px;
            display: inline-block;
            text-decoration: none;
            font-size: 14px;
            transition: background 0.15s ease-in-out, transform 0.05s ease-in-out;
        }
        button:hover, .botao:hover { background: #1a252f; }
        button:active, .botao:active { transform: translateY(1px); } /* pequeno
"afundar" ao clicar, feedback tátil visual */

        /* ----- Barra de ações (topo do dashboard) ----- */
        .barra-acoes { margin-bottom: 16px; display: flex; flex-wrap: wrap; gap:
8px; }
        .barra-acoes a { margin-top: 0; } /* sobrescreve o margin-top padrão
do .botao dentro da barra */

        /* ----- Estado vazio (nenhum produto cadastrado) ----- */
        .estado-vazio { text-align: center; color: #8a94a3; padding: 10px 0; font-
size: 14px; }
    </style>
</head>
<body>
    <header>
        <span class="marca">📦 Controle de Estoque</span>
        <nav>
            {% if session.get('usuario_id') %}

```

```

        <span>{{ session.get('usuario_nome') }}
    ({{ session.get('perfil') }})</span>
        <a href="{{ url_for('dashboard') }}">Início</a>
        <a href="{{ url_for('relatorios') }}">Relatórios</a>
        <a href="{{ url_for('logout') }}">Sair</a>
    {% endif %}
</nav>
</header>
<main>
    {% with mensagens = get_flashed_messages(with_categories=true) %}
        {% for categoria, texto in mensagens %}
            <div class="flash {{ categoria }}">{{ texto }}</div>
        {% endfor %}
    {% endwith %}
    {% block conteudo %}{% endblock %}
</main>
</body>
</html>

```

A.12 templates/dashboard.html

```
{% extends "base.html" %}
{% block conteudo %}
<h2>Estoque Atual</h2>

<div class="barra-acoes">
  <a class="botao" href="{{ url_for('novo_produto') }}">Novo Produto</a>
  <a class="botao" href="{{ url_for('novo_fornecedor') }}">Novo Fornecedor</a>
  {% if session.get('perfil') == 'admin' %}
    <a class="botao" href="{{ url_for('novo_usuario') }}">Cadastrar Usuário</a>
  {% endif %}
  <a class="botao" href="{{ url_for('dashboard') }}">Atualizar Lista</a>
</div>

<table>
  <tr>
    <th>ID</th><th>Nome</th><th>Código</th><th>Categoria</th><th>Fornecedor</th>
    <th>Qtd. Atual</th><th>Qtd. Mínima</th><th>Preço Unit.</th><th>Ações</th>
  </tr>
  {% for p in produtos %}
    <tr class="{{ 'alerta' if p['quantidade_atual'] < p['quantidade_minima'] else '' }}">
      <td>{{ p['id_produto'] }}</td>
      <td>{{ p['nome'] }}</td>
      <td>{{ p['codigo'] }}</td>
      <td>{{ p['categoria_nome'] or '-' }}</td>
      <td>{{ p['fornecedor_nome'] or '-' }}</td>
      <td>{{ p['quantidade_atual'] }}</td>
      <td>{{ p['quantidade_minima'] }}</td>
      <td>R$ {{ "%.2f"|format(p['preco_unitario']) }}</td>
      <td>
        <a href="{{ url_for('movimentacao', id_produto=p['id_produto'],
        tipo='entrada') }}">Entrada</a> |
        <a href="{{ url_for('movimentacao', id_produto=p['id_produto'],
        tipo='saida') }}">Saída</a>
      </td>
    </tr>
  {% else %}
    <tr><td colspan="9" class="estado-vazio">Nenhum produto cadastrado ainda.
    Clique em "Novo Produto" para começar.</td></tr>
  {% endfor %}
</table>
{% endblock %}
```

APÊNDICE B – SCRIPT SQL DA ESTRUTURA DO BANCO DE DADOS

Este apêndice apresenta o script SQL completo (database/schema.sql) com a definição das cinco tabelas do banco de dados relacional utilizado pela aplicação (SQLite), incluindo chaves primárias, chaves estrangeiras e restrições de integridade (NOT NULL, UNIQUE e CHECK). O mesmo arquivo também é entregue separadamente dentro do arquivo compactado do projeto (Projeto_Python_Control_Estoque.zip), conforme exigido na Parte Prática do trabalho.

```
-- =====
-- Script SQL - Estrutura do banco de dados
-- Sistema de Controle de Estoque - Development with Python
-- Banco de dados: SQLite
-- Gerado a partir de database/db.py (funcao criar_tabelas)
-- =====

-- Tabela: usuario
CREATE TABLE usuario (
    id_usuario      INTEGER PRIMARY KEY AUTOINCREMENT,
    nome            TEXT NOT NULL,
    login           TEXT NOT NULL UNIQUE,
    senha_hash      TEXT NOT NULL,
    perfil          TEXT NOT NULL CHECK (perfil IN ('admin', 'comum')),
    data_cadastro   TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Tabela: categoria
CREATE TABLE categoria (
    id_categoria    INTEGER PRIMARY KEY AUTOINCREMENT,
    nome            TEXT NOT NULL UNIQUE
);

-- Tabela: fornecedor
CREATE TABLE fornecedor (
    id_fornecedor   INTEGER PRIMARY KEY AUTOINCREMENT,
    nome            TEXT NOT NULL,
    cnpj            TEXT NOT NULL UNIQUE,
    telefone        TEXT
);

-- Tabela: produto
CREATE TABLE produto (
    id_produto      INTEGER PRIMARY KEY AUTOINCREMENT,
    nome            TEXT NOT NULL,
    codigo          TEXT NOT NULL UNIQUE,
    id_categoria    INTEGER,
    id_fornecedor   INTEGER,
    quantidade_atual INTEGER NOT NULL DEFAULT 0,
    quantidade_minima INTEGER NOT NULL DEFAULT 0,
    preco_unitario  NUMERIC(10,2) NOT NULL DEFAULT 0,
    FOREIGN KEY (id_categoria) REFERENCES categoria (id_categoria),
    FOREIGN KEY (id_fornecedor) REFERENCES fornecedor (id_fornecedor)
);

-- Tabela: movimentacao
CREATE TABLE movimentacao (
    id_movimentacao INTEGER PRIMARY KEY AUTOINCREMENT,
    id_produto       INTEGER NOT NULL,
    id_usuario       INTEGER NOT NULL,
    tipo            TEXT NOT NULL CHECK (tipo IN ('entrada',
'saida'))),
    quantidade       INTEGER NOT NULL,
```

```
    preco_unitario_momento NUMERIC(10,2) NOT NULL DEFAULT 0,  
    data_hora              TEXT DEFAULT CURRENT_TIMESTAMP,  
    observacao             TEXT,  
    FOREIGN KEY (id_produto) REFERENCES produto (id_produto),  
    FOREIGN KEY (id_usuario) REFERENCES usuario (id_usuario)  
);  
  
-- Usuario administrador padrao criado na primeira execucao:  
-- login: admin | senha: admin123 (hash SHA-256 armazenado, nao a senha em texto  
puro)
```

APÊNDICE C – SINCRONIZAÇÃO EM TEMPO REAL (NÍVEL 2): DESAFIOS DE IMPLEMENTAÇÃO

Este apêndice complementa a Seção 2.5 (Arquitetura de duas interfaces), detalhando o que seria necessário para evoluir a sincronização hoje implementada — "sob demanda", isto é, ao clicar em "Atualizar Lista" no Tkinter ou recarregar a página no navegador — para uma sincronização em tempo real, na qual uma alteração feita em uma interface aparece automaticamente na outra, sem qualquer ação manual do usuário. Essa análise foi elaborada em nível conceitual e arquitetural, sem implementação de código, para demonstrar o entendimento das técnicas envolvidas e justificar a decisão de manter a sincronização sob demanda no escopo deste trabalho.

C.1 Técnicas possíveis para sincronização em tempo real

- a) polling (sondagem periódica): um script JavaScript no navegador consultaria uma rota do Flask a cada poucos segundos (por exemplo, a cada 3 segundos), perguntando se algo mudou desde a última consulta; do lado do Tkinter, um temporizador chamado por `self.after()` faria o mesmo, reconsultando o banco periodicamente. É a técnica mais simples de implementar, mas gera tráfego de rede/consultas ao banco mesmo quando nada mudou, e a atualização nunca é instantânea — só tão rápida quanto o intervalo configurado;
- b) WebSockets (ex.: biblioteca Flask-SocketIO): o servidor mantém uma conexão aberta e bidirecional com cada cliente conectado (navegadores e, potencialmente, o próprio Tkinter como um cliente Socket.IO) e "empurra" (push) o evento de mudança assim que ele ocorre, sem que o cliente precise perguntar. É a técnica mais responsiva, mas exige um servidor com suporte a operações assíncronas (tipicamente as bibliotecas `eventlet` ou `gevent`), além de uma biblioteca cliente extra para o lado do Tkinter conseguir participar dessa comunicação;
- c) observador de arquivo (watchdog): um processo específico ficaria monitorando alterações no arquivo `database/estoque.db` no sistema de arquivos e dispararia um evento quando o arquivo fosse modificado. É tecnicamente mais simples que WebSockets, mas menos preciso, pois não informa o que mudou — apenas que "algo" mudou —, exigindo que a interface recarregue toda a listagem a cada notificação.

C.2 Principais desafios de implementação identificados

- a) modelos de concorrência incompatíveis entre as duas interfaces: o Flask, no modo padrão, opera em ciclo requisição-resposta (cada requisição HTTP é tratada e finalizada); o Tkinter, por sua vez, roda em um laço de eventos de thread única (mainloop). Integrar uma técnica de tempo real exigiria, no lado do Tkinter, executar a comunicação de rede (seja um cliente WebSocket, seja um polling) em uma thread separada da interface gráfica, comunicando-se com a thread principal por meio de uma fila (queue) thread-safe — do contrário, a interface travaria ("congelaria") enquanto aguardasse dados da rede;
- b) necessidade de um mecanismo de notificação de mudanças: hoje, a camada models/ apenas grava no banco e retorna; para viabilizar eventos em tempo real, seria necessário que essa camada também publicasse um evento ("produto X foi atualizado") para um canal compartilhado entre os processos — por exemplo, via Flask-SocketIO no lado do servidor —, o que exigiria repensar a separação de responsabilidades entre models/ (regra de negócio) e a camada de notificação (infraestrutura);
- c) controle de concorrência na gravação: com duas interfaces podendo alterar os mesmos dados de forma praticamente simultânea, cresce o risco de condições de corrida (por exemplo, dois usuários registrando saída do mesmo produto ao mesmo tempo, cada um calculando o novo saldo a partir de uma leitura já desatualizada). Mitigar esse risco exigiria mecanismos de controle de concorrência otimista (como uma coluna de versão incrementada a cada UPDATE, conferida antes de gravar) ou bloqueios explícitos de linha, temas normalmente tratados em bancos de dados cliente-servidor mais robustos;
- d) limitações do SQLite para escrita concorrente: o SQLite é um banco de arquivo único, adequado ao escopo deste projeto acadêmico, mas com suporte limitado a múltiplos processos gravando simultaneamente (mesmo em modo WAL — Write-Ahead Logging). Uma solução de tempo real em produção tenderia a exigir a migração para um banco cliente-servidor, como PostgreSQL ou MySQL, ambos citados como alternativas válidas de banco de dados no enunciado do desafio;
- e) aumento da complexidade operacional: a solução de tempo real introduziria processos adicionais (servidor assíncrono, ou processo observador de arquivo), mais pontos de falha, e maior dificuldade de depuração e de demonstração em um contexto de avaliação acadêmica — o que reforça a decisão deste trabalho de documentar a técnica em nível conceitual, sem incorporá-la ao código entregue.

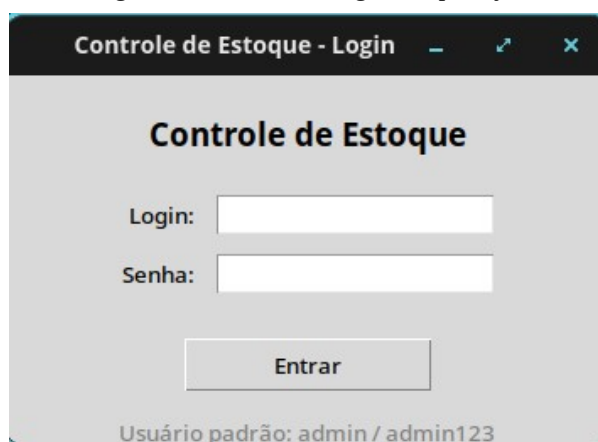
C.3 Conclusão da análise

A sincronização sob demanda, efetivamente implementada neste projeto (Seção 2.5), atende ao requisito de que uma atualização feita em uma interface seja refletida na outra, respeitando o escopo e a complexidade esperados de um trabalho introdutório de Python. A sincronização em tempo real (Nível 2) foi estudada em profundidade suficiente para identificar as técnicas candidatas (polling, WebSockets e observação de arquivo) e os desafios reais de arquitetura que ela introduziria — sobretudo a necessidade de threads separadas no Tkinter, um mecanismo de notificação de eventos entre processos, e controle de concorrência na escrita —, permanecendo registrada aqui como direção natural de evolução do projeto.

APÊNDICE D – CAPTURAS DE TELA DA APLICAÇÃO EM EXECUÇÃO

Este apêndice apresenta capturas de tela (prints) da aplicação de controle de estoque em execução real, tanto na interface desktop (Tkinter) quanto na interface web (Flask), evidenciando o funcionamento efetivo das funcionalidades descritas ao longo deste trabalho: autenticação, cadastro de fornecedor e de produto com geração automática de código, validação de campos, registro de movimentação de estoque, o alerta visual de quantidade abaixo do mínimo, e a sincronização entre as duas interfaces por meio do banco de dados compartilhado.

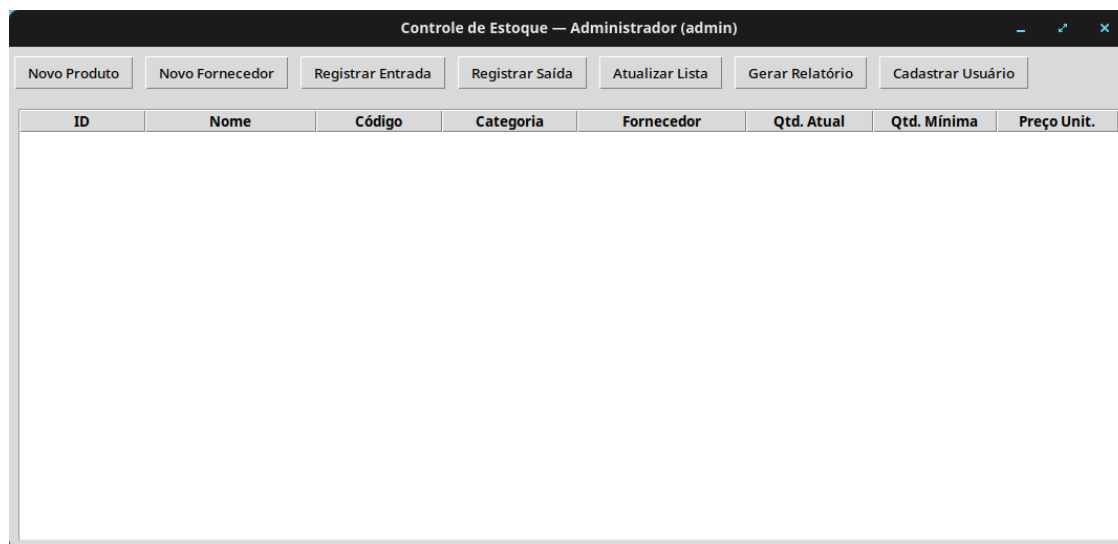
Figura D.1 — Tela de login da aplicação



A captura de tela mostra uma janela de login com o título 'Controle de Estoque - Login'. O conteúdo principal é o 'Controle de Estoque' em negrito. Abaixo, há campos para 'Login:' e 'Senha:', cada um com um campo de entrada de texto. Um botão 'Entrar' está centralizado abaixo dos campos. Na base da janela, há uma dica de usuário: 'Usuário padrão: admin / admin123'.

Fonte: elaborado pelo autor (2026).

Figura D.2 — Tela principal, exibida após o login, ainda sem produtos cadastrados



A captura de tela mostra a interface principal do administrador. O título da janela é 'Controle de Estoque — Administrador (admin)'. No topo, há uma barra de menu com botões: 'Novo Produto', 'Novo Fornecedor', 'Registrar Entrada', 'Registrar Saída', 'Atualizar Lista', 'Gerar Relatório' e 'Cadastrar Usuário'. Abaixo, há uma tabela com as seguintes colunas: 'ID', 'Nome', 'Código', 'Categoria', 'Fornecedor', 'Qtd. Atual', 'Qtd. Mínima' e 'Preço Unit.'. A tabela está vazia, indicando que não há produtos cadastrados.

Fonte: elaborado pelo autor (2026).

Figura D.3 — Formulário de cadastro de fornecedor preenchido

Controle de Estoque — Administrador (admin)

Novo Produto Novo Fornecedor Registrar Entrada Registrar Saída Atualizar Lista Gerar Relatório Cadastrar Usuário

ID	Nome	Código	Categoria	Fornecedor	Qtd. Atual	Qtd. Mínima	Preço Unit.
Novo Fornecedor Nome* Distribuidora ABC CNPJ* 12.345.678/0001-90' Telefone Salvar							

Fonte: elaborado pelo autor (2026).

Figura D.4 — Formulário de cadastro de produto, com o aviso de que o código é gerado automaticamente

Novo Produto

O código do produto será gerado automaticamente ao salvar.

Categoria Alimentos

Fornecedor Distribuidora ABC (12.34!)

Nome* Banana

Qtd. Inicial* 100

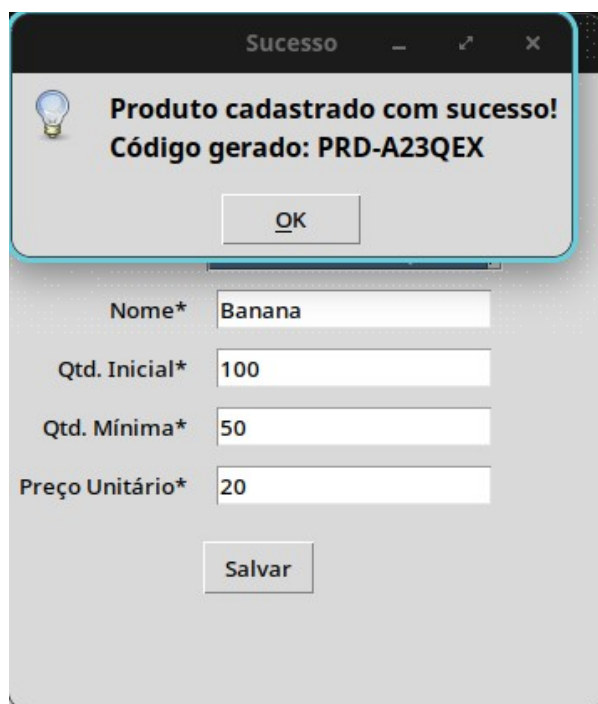
Qtd. Mínima* 50

Preço Unitário* 20

Salvar

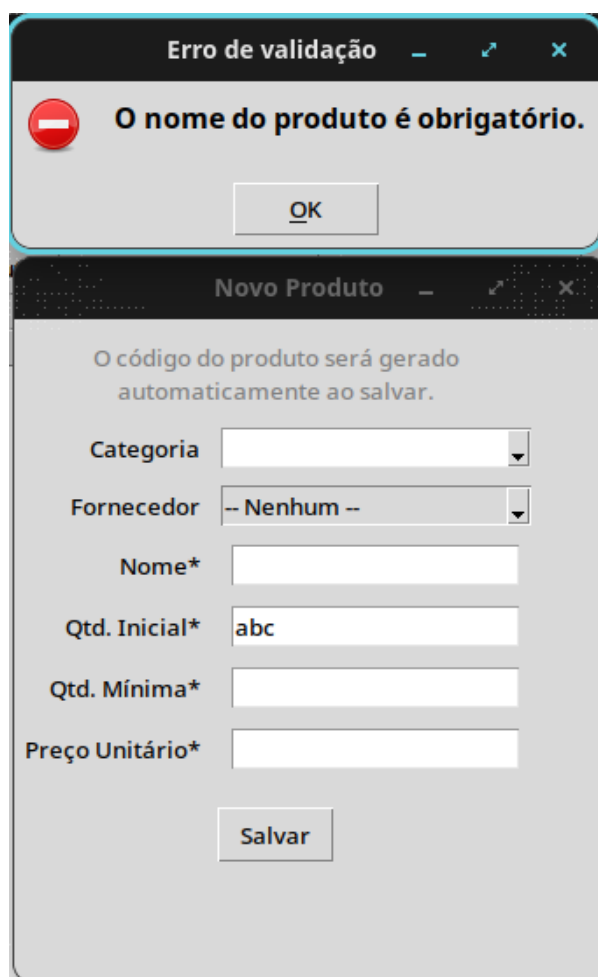
Fonte: elaborado pelo autor (2026).

Figura D.5 — Confirmação de cadastro exibindo o código gerado automaticamente (PRD-A23QEX)



Fonte: elaborado pelo autor (2026).

Figura D.6 — Mensagem de erro de validação ao tentar cadastrar um produto sem nome informado



Fonte: elaborado pelo autor (2026).

Figura D.7 — Confirmação de movimentação de estoque registrada com sucesso



Fonte: elaborado pelo autor (2026).

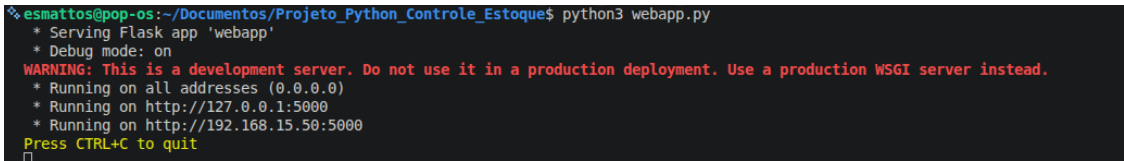
Figura D.8 — Listagem de estoque com o produto "Banana" destacado em vermelho, após a quantidade atual (40) ficar abaixo da quantidade mínima configurada (50)

Controle de Estoque — Administrador (admin)							
Novo Produto		Novo Fornecedor		Registrar Entrada		Registrar Saída	
Atualizar Lista		Gerar Relatório		Cadastrar Usuário			
ID	Nome	Código	Categoria	Fornecedor	Qtd. Atual	Qtd. Mínima	Preço Unit.
1	Banana	PRD-A23QEX	Alimentos	Distribuidora ABC	40	50	R\$ 20.00

Fonte: elaborado pelo autor (2026).

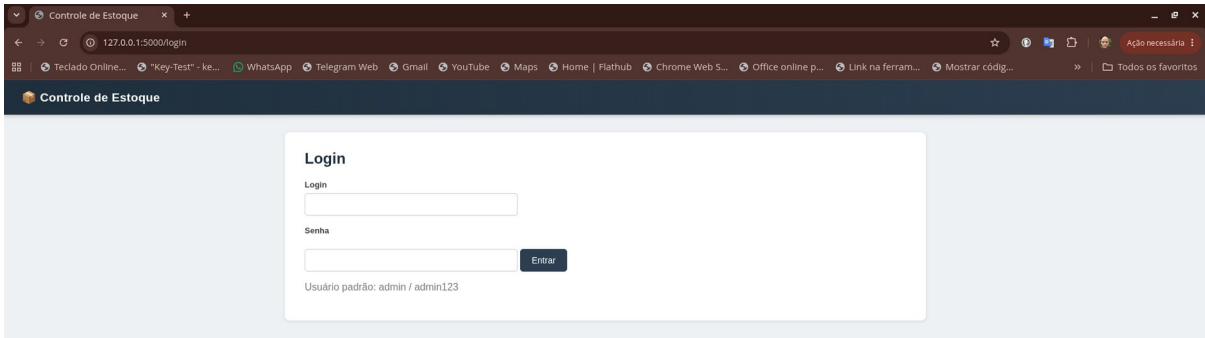
As figuras a seguir demonstram a interface web (Flask) da mesma aplicação, executada a partir do arquivo webapp.py em um segundo terminal, evidenciando a arquitetura de duas interfaces descrita na Seção 2.5.

Figura D.9 — Terminal executando o comando "python3 webapp.py", iniciando o servidor Flask de desenvolvimento



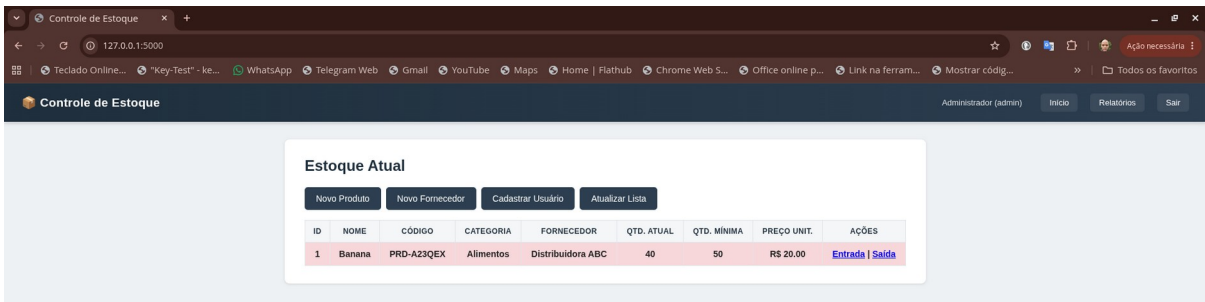
Fonte: elaborado pelo autor (2026).

Figura D.10 — Tela de login da aplicação, acessada pelo navegador em 127.0.0.1:5000



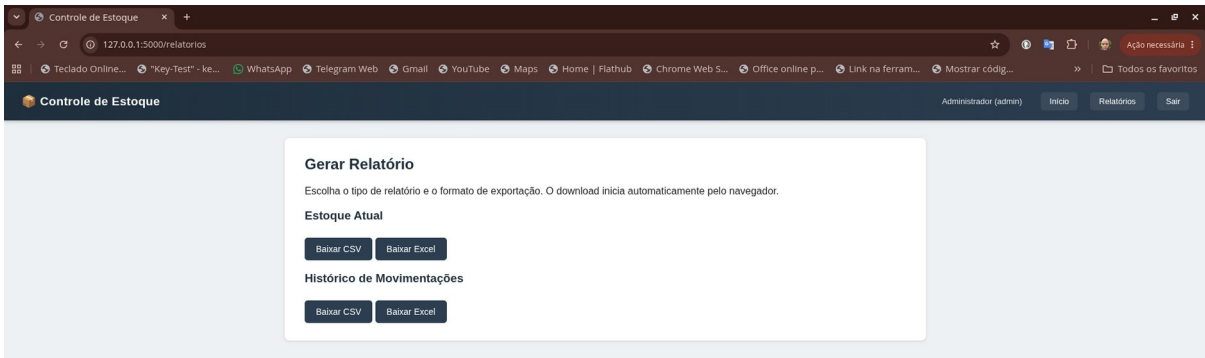
Fonte: elaborado pelo autor (2026).

Figura D.11 — Painel web exibindo o produto "Banana" (código PRD-A23QEX, quantidade 40/50) com o mesmo alerta visual de estoque baixo já registrado na Figura D.8, comprovando a sincronização entre as interfaces desktop e web por meio do banco de dados compartilhado



Fonte: elaborado pelo autor (2026).

Figura D.12 — Página de geração de relatórios da interface web, com opções de exportação em CSV e Excel para o estoque atual e o histórico de movimentações



Fonte: elaborado pelo autor (2026).